

Danmarks
Tekniske
Universitet



41801 PROJECT WORK

GRUPPE NR: 3

AI ROBOTIC ARM

Benjamin Kauffmann Vener - s235152

Daniel Malik Mapouyat - s235180

Jonathan Schnack Ahrenkilde - s235149

Mads Tornby Christoffersen - s232624

Nick Gholami - s235175

Vejleder: Konstantinos Poullos

December 12, 2025

Abstract

The purpose of this project is to eliminate manual waste sorting in low-resource areas. This paves the way for the development of an AI-driven robotic arm with servo motors, 3D-printed components, and image recognition that automates the sorting process. A SWOT analysis is conducted, and a Unique Selling Point (USP) is identified as a basis for the problem analysis and market research. Concept selection is carried out with special focus on a morphological chart and the evaluation of stepper vs. servo motors. CAD design is performed with a specific focus on tolerances and mechatronic setup. The implementation of OpenAI and inverse kinematics via IKPy contributes to the transition from manual sorting to a sustainable, user-friendly, and cost-effective solution. The obtained results show that the robotic arm has good repeatability accuracy, enabling the classification system to quickly and precisely distinguish E-waste. Furthermore, limitations in URDF and calibration were addressed through software adaptations. Overall, the system serves as a proof-of-concept with future potential for increased robustness and scalability.

Forord

Projektet er udført på DTU som en del af 41801 Fagprojekt i sommeren 2025. Gruppen består af Daniel, Benjamin, Mads, Nick og Jonathan. Tak til gruppens vejleder, Konstantinos Poullos, for faglig sparring og værdifuld feedback. En særlig tak skal også lyde til DTU-faciliteterne – især Skylab – for adgang til 3D-printere, værksted og relevant software.

Gennem projektet har vi lært at kombinere mekanik, elektronik og kunstig intelligens. Den største udfordring har været arbejdet med inverse kinematik og kalibrering af servomotorer samt deres præcise placering. Dette førte dog også til det største læringsudbytte i form af samarbejde, fejlfinding og integration mellem hardware og software.

Indholdsfortegnelse

1	Introduktion	1
1.1	Problemanalyse	1
1.2	Problemformulering	1
2	Markedsanalyse og grundspecifikation	2
2.1	Nuværende Marked	2
2.1.1	State of The Art	2
2.2	Unique Selling Point	3
2.3	SWOT ANALYSE	3
2.4	Grundspecifikationer	5
3	Idegenerering og konceptudvalg	5
3.1	Indledende Journey map	5
3.2	Designmuligheder, frihedsgrader og akser	6
3.3	Morfologisk skema og koncepter	6
3.3.1	Non Newtonian Gripper(GUL)	7
3.3.2	Stepper Motor Mekanisk Gripper(RØD)	7
3.3.3	Servo Motor Mekanisk Gripper(BLÅ)	7
3.3.4	Udvælgelseskriterier	7
3.4	Styrkeberegninger	7
3.5	Designbeslutninger og overgang til konstruktion	8
4	Konstruktionsfase	9
4.1	Overordnet opsætning	9
4.2	CAD og Prototypedesign	10
4.2.1	Prototype 1 – Steppermotor	10
4.2.2	Prototype 2 – Servomotor	10
4.2.3	3D-print og tolerancer	10
4.3	Beskrivelse af det mekatroniske setup	10
4.3.1	Strømforsyning	12
4.3.2	Motorspecifikationer	12
4.4	Software	12
4.4.1	Kamera og billedbehandling	12
4.4.2	Samarbejde med OpenAI	13
4.4.3	URDF-fil	13
4.4.4	Invers kinematics	15
4.4.5	Kompensation for kalibrering i servomotorer	17
4.4.6	Overordnet system	18

5 Test af systemets funktionelle egenskaber	19
5.1 Positionsnojagtighed	19
5.2 Gentagelsesnojagtighed	20
5.3 Test af løfteevne	21
5.4 Test af AI-klassifikation	22
6 Videreudviklingsmuligheder	24
7 Diskussion	24
8 Konklusion	26
9 Appendix A	28
9.1 Bill of Materials (BOM) – Robotic Arm Assembly	28
9.2 Arbejdstegetninger	29
10 Appendix B	30
10.1 Forsøgsresultater	30
10.1.1 Præcision	30
10.1.2 Gentagelsestest	30
10.1.3 Klassifikation	31
11 Appendix C	32
11.1 Prototyper	32
12 Appendix D	33
12.1 Koder	33
13 Appendix E	58
13.1 Styrke Udregninger - Maple	58
14 Appendix F	59
14.1 Tekstprompt til OpenAI	59
15 Appendix G	60
16 Appendix H	61

1 Introduktion

1.1 Problemanalyse

Den teknologiske udvikling har på globalt plan medført betydelige forbedringer i både samfundsmæssige og individuelle vilkår. Denne udvikling har dog også en mørk bagside: Den eksplosive vækst i affald. I takt med, at nye produkter erstatter gamle med kortere intervaller, ender millioner af ton affald på lossepladser – ofte i lavindkomstlande, hvor affaldet udskibes uden en bæredygtig plan for behandling og sortering.

I disse lande – typisk i dele af Afrika og Asien – foregår affalds-håndtering under uformelle og farlige forhold. Sorteringen sker primært manuelt, hvor arbejderen udsættes for giftige materialer, tungmetaller og personfarlige komponenter, alt imens genanvendelsesraten for værdifulde materialer forbliver lav.[1]

Samtidig er autonome teknologier og avanceret robotik enten for dyre eller for komplekse at implementere i disse omgivelser. Der mangler løsninger, som både er:

- Prisvenlige
- Let anvendelige
- Fleksible og adaptive til forskellige typer affald
- Og vigtigst af alt: Sikre at operere under lavressourceforhold

Problematikken ses også i Danmark, hvor nye studier viser at sorteringsproblemet koster danskere penge. Antagelsen om at den almene dansker sorterer, og vigtigst, sorterer korrekt, koster dyrt i kommunerne. Hvis der ikke sorteres korrekt fra starten, skal der sorteres i affaldcentre, hvor der er mangel på gode løsninger til det bredde udvalg af affald som kommer fra forbrugeren. [2]

1.2 Problemformulering

Affald håndteres ofte under farlige forhold i lavressourceområder, hvor der mangler sikre og tilgængelige teknologiske løsninger. Der er derfor behov for en enkel og effektiv tilgang til sortering af arbitrært affald.

Hvordan kan man designe en teknologisk løsning, der muliggør en mere sikker, effektiv og målrettet sortering af arbitrært affald – under hensyntagen til krav om lav pris, brugervenlighed og evne til at håndtere varierende typer af affald uden behov for avanceret infrastruktur?

2 Markedsanalyse og grundspecifikation

2.1 Nuværende Marked

Der er opstået et behov for en ny generation af teknologier, der kan automatisere sortering og komponentgenvinding uden krav om avanceret infrastruktur – og samtidig sikre arbejdsmiljøet og beskytte naturen omkring lossepladserne.

Denne løsning taler direkte ind i flere af FN's verdensmål for bæredygtig udvikling:

- **SDG 3 – Sundhed og trivsel:** Ved at reducere eksponeringen for giftige materialer og farlige arbejdsforhold forbedres arbejdssikkerheden for mennesker involveret i uformel affaldshåndtering.
- **SDG 8 – Anstændige jobs og økonomisk vækst:** Skaber mulighed for nye tekniske arbejdspladser og forbedrer produktiviteten i lavressourceområder gennem teknologisk støtte.
- **SDG 9 – Industri, innovation og infrastruktur:** Introducerer overkommelig, fleksibel automationsteknologi som en bæredygtig og inkluderende innovationsløsning.
- **SDG 12 – Ansvarligt forbrug og produktion:** Fremmer genbrug og effektiv ressourceudnyttelse ved at optimere genanvendelsen af specialiserede og eller svært sorterede komponenter.
- **SDG 13 – Klimaindsats:** Reduktion af miljøbelastning fra ukontrolleret affald bidrager til mindre forurening og klimaskadelige effekter.[3]

2.1.1 State of The Art

På globalt plan er der allerede udviklet flere teknologiske løsninger, der alle automatisere sortering og behandling af affald. Dog er langt de fleste af disse systemer udviklet med fokus på højindkomstlande og store industrielle anlæg, hvilket gør dem uegnede til lavressourceområder med decentral affaldshåndtering.

Virksomheder som **AMP Robotics**, **ZenRobotics** og **EverestLabs**[4, 5, 6] anvender billedgenkendelse og robotteknologi til at sortere affald i høj hastighed og på stor skala. Disse systemer er dog primært designet til ensartede miljøer og store genbrugsstationer, og de mangler fleksibilitet i forhold til at håndtere den store variation og uforudsigelighed, som ofte kendetegner affald i udviklingslande.

Andre aktører som **HEBI Robotics** og **Universal Robots** tilbyder modulære robotsystemer, som kan tilpasses forskellige opgaver, herunder affaldshåndtering. Disse kræver dog typisk teknisk ekspertise og adgang til infrastruktur, som ofte ikke er tilgængelig i lavindkomstområder.

Fælles for næsten alle konfigurationer ses det, at "Pick And Place" -robotter bruges. Denne konfiguration opstår typisk, da affald ofte varierer i geometri og masse og dermed kræver en taktil opsamling. På trods af teknologiske fremskridt er der stadig et klart hul i markedet for en **lavpris, adaptiv og**

brugervenlig robotløsning, der kan operere effektivt i miljøer med begrænsede ressourcer og høj variation i affaldstyper. Dette er præcis det hul, som den potentielle løsning søger at udfylde. Illustreres dette område ses det som det grønne felt i Figur 1.

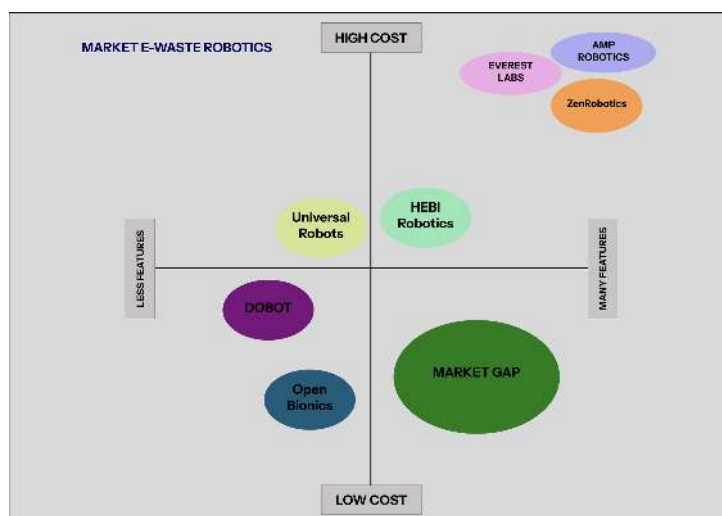


Figure 1: Market Gap

2.2 Unique Selling Point

En grundlæggende problematik med de fleste robotiske sorteringssystemer er, at de oftest er specialiseret til én type arbejde. Denne mangel på adaptation gør dem ukampdygtige i mange af verdens landfills. Derudover ses begrænsninger på sorteringer indenfor kategorier. En voksende problematik er E-waste som indebærer affald fra den elektroniske sektor. Her kræves der specialiseret sortering i et dybere niveau end blot den overordnede kategori. Der vil derfor være en stor fordel i at udvikle løsninger der kan differentiere blandt elektroniske komponenter. Ved at implementere Artificial Intelligens kan denne klassifikation laves helt uden menneskelig intervention.

2.3 SWOT ANALYSE

I takt med at løsning skal integreres på et mættet marked er det kritisk at overveje hvorvidt løsningen er feasible, viable and desirable. Hertil er der udført en SWOT-analyse for at overveje problemstillinger og styrker ved markedet og produktet.

Interne forhold	
Strengths	Weaknesses
- Attraktiv Prispunkt - Modulær produkt (Plug and Play) - Let at vedligeholde - Bred funktionalitet - Kræver minimal erfaring	- Small scale solution - Præcision
Eksterne forhold	
Opportunities	Threats
- Vækstende Marked - FN's verdensmål promoverer løsninger som denne - Markedet øges i lavindkomstlande	- Stor konkurrence - Lovgivningsaspekter - Tredje parts komponenter

Strengths:

Produktets største styrke ligger i dets attraktive pris og modulære design, hvilket gør det nemt at tilpasse og implementere i mange forskellige miljøer. Systemet kræver minimal erfaring at betjene og er let at vedligeholde, hvilket gør det ideelt til lavressourceområder og hurtigt skiftende arbejdssituationer. Den brede funktionalitet giver mulighed for fleksibel anvendelse uden store ændringer i opsætning.

Weaknesses:

Det kan være en udfordring at skalere løsningen til opgaver, der kræver høj nøjagtighed eller større mekanisk styrke, hvilket begrænser dens anvendelse i visse mere avancerede eller omfattende sammenhænge.

Opportunities:

Markedet for fleksible og sikre robotløsninger vokser, især i lavindkomstlande, hvor behovet for overkommelige og brugervenlige systemer er stigende. Samtidig skaber FN's verdensmål et øget på fokus på teknologier, der fremmer bæredygtig affaldshåndtering og lokal automatisering.

Threats:

På trods af markedspotentialet eksisterer der allerede betydelig konkurrence fra både etablerede aktører og nye startups. Desuden kan afhængighed af tredjeparts-komponenter og komplekse lovgivningskrav i forskellige regioner bremse udbredelsen.

2.4 Grundspecifikationer

Med markedet og problemstillingen kortlagt kan der udvikles en grundspecifikation. Specifikationen er udviklet over flere iterationer efter undersøgelse af markedet og de specifikke arbejdsopgaver en robotarm kunne være underlagt.

EMNE (f.eks.)	KRAV(Demand)	KRITERIER(Wish)	BEMÆRKNING
Akser/Frihedsgrader	Minimum 4 frihedsgrader		For at løfte 3-dimensionelt
	Minimum arbejdende i 3 rumlige dimensioner		
Gribemekanisme	Skal kunne gribe abstrakte former		Former begrænses såvidt til typiske elektroniske komponenter
	Regulerer tryk ud fra objektets hårdhed		
	Skal kunne gribe et objekt med bredde 15cm		
	Frictionskoefficient mellem fingre og objekt skal være tilstrækkelig		
	Vandret kontaktflade mellem fingre og Objekt		
Sensor	Tactile Sensor i fingrespider til regulering af tryk		
	Distancesensor til måling af Objektets højde		
Kamera	Bred FOV - Skal kunne se hele arbejdsområdet	Geme uden at hæve kamera unødigt meget	
	Høj opløsning - Skal kunne definere objekter		
	Autofokuser		
	Skal være integrerbar med Raspberry Pi		
Løftemekanisme	Skal kunne arbejde i forskellige lys-områder	Skal kunne opereres i mørke områder	
	Løfte eller sænke et objekt i lodret akse på mindst 700g	Løfte eller sænke til et specifikt sted	
Software	Skal uploade billede til Open AI's - Image Recognition Model	Geme uden stor latency	
	Skal kunne identificere og estimere objektets placering og type		
	Skal definere et estimat af tryk tilstrækkelig for at løfte et objekt		
	Skal samle informationer fra forskellige sensorer og kamera til et samlet script		
Strømforsyning	Skal levere 2 forskellige spændinger til forskellige motor		
	Skal levere højt antal Ampere for at give holding torque og kunne køre samtlige motorer samtidigt	Som udgangspunkt omkring 15-20 A til alle	
	Konvertere AC til DC		

Figure 2: Grundspecifikation

3 Idegenerering og konceptudvalg

Til udvidelsen af potentielle løsninger udforskes løsningsrummet på baggrund af de specifikationer defineret i forrige afsnit. Heri vil forskellige designs og konfigurationer for robotarmen udforskes. En detaljeret beskrivelse af udvælgelsesprocessen og de kriterier og overvejelser som har lagt for den endelige konfiguration af arm og hånd. Hertil kommer ikke allerede beskrevne begrænsninger som forekommer ved armen og håndens elektroniske komponenter og mekaniske styrke til udtryk.

3.1 Indledende Journey map

Løsningen har til formål at effektivisere og automatisere affaldssortering i lavressourceområder. Processen er visualiseret i et journey map, som fremgår af Figur 3.

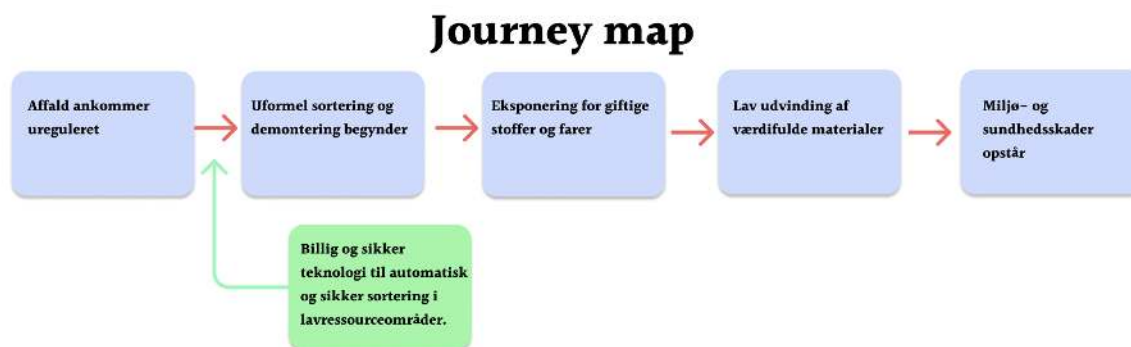


Figure 3: Journey map

3.2 Designmuligheder, frihedsgrader og akser

For at løse problemstillingen var det vigtigt at armen bevægede sig i det 3-dimensionelle rum. Dette medfører at armens konfiguration må udvælges så dette opnås. En robotarm kan i sin natur gøres næsten uendelig kompleks. En ofte set konfiguration er givet ved 3 frihedsgrader (3 uafhængigt styrede led) hvis hensigt er, at opnå en 3 akset bevægelse.

3.3 Morfologisk skema og koncepter

Til udvælgelsen af forskellige designs blev en brainstorm og morfologisk skema udført. Hensigten var at udbrede og herefter indskrænke løsningsrummet strategisk ved brug af double diamond metoden. Hertil blev 3 konfigurationer udledt. Se Figur 20.

Funktioner	Solution 1	Solution 2	Solution 3	Solution 4	Solution 5
Armbbevægelse	Stepper Motorer 	Pulley system	Servomotorer 	Hydraulik	Pneumatiske aktuatorer
Lukkemekanisme Hånd	Luftpude	Mekanisk Lukning med fingre 	Artificial Muscle Material	Opsugning	Non Newtonian fluid 
Image Recognition	Open AI API 	yolo V8 Image Recognition	Udvikle Egen 	Klassifikations model	
Maksbegrænsning af Tryk	Strain af motorer (strømspænding)	Tryksensorer 	Image analysis	Strain sensor på objekt	Måling af friktion 

Figure 4: Morfologisk Skema

De 3 konfigurationer af armen er defineret og deres respektive fordele og ulemper blev nu udforsket.

3.3.1 Non Newtonian Gripper(GUL)

En konfiguration af hånd og arm var udtænkt til at gøre brug af stepper motorer, med en hånd der omfavner emner med non newtonian fluid, et selvudviklet software og at betragte friktionen mellem emne og hånd til at definere løftetryk.

Denne konfiguration blev kasseret på baggrund af kompleksiteten og ønsket om en let vedligeholdelse samt frygten for at væskeposen kunne punktere ved opsamling af spidse emner.

3.3.2 Stepper Motor Mekanisk Gripper(RØD)

En standard konfiguration med mekanisk gribefunktion og stepper motorer blev udvalgt. Denne løsning lovede godt i og med stepper motorer generelt er præcise og softwaremæssigt nemme og arbejde med. Et kriterie for armen er et simpelt set-up med minimal vedligeholdelse. stepper motorer er optimale til netop dette og sikre at arbejdsopgaver er let genskabelige og præcisionen er høj. Denne konfiguration blev valgt at arbejde videre med.

3.3.3 Servo Motor Mekanisk Gripper(BLÅ)

Servomotor konfigurationen var den sidste løsning som blev udtænkt gennem proceduren for det morfologiske skema. Denne løsning var udtænkt i tilfælde af at stepper motorerne ikke havde tilstrækkeligt moment til at udføre typiske arbejdsopgaver for armen. Generelt er der større omkostninger og softwaremæssige komplikationer forbundet med servomotorer. Denne konfiguration ville derfor agere som back-up til stepper motorer.

3.3.4 Udvælgelseskriterier

For at kunne vælge mellem Servo- og Stepper motorer udførtes en matematisk analyse af det moment der opstår ved drift af armen. Dette ses i kommende afsnit.



Figure 5: Udkast Design

En grov skitse af konfigurationen udvundet fra det morfologiske skema blev tegnet og ses i Figur 5.

3.4 Styrkeberegninger

En klar begrænsning der tidligt i projektet blev gjort klar, var det moment der var krævet af motorene. Et klart kriterie til Robot-arm og -hånd var at komponenterne var omkostningsvenlige og modulare. Det medfører også at disse komponenter måtte vælges med omhu for at sikre at de overholdte de specifikationer som var krævet. Generelt for stepper motorer vil bevægelsen være glat, og utroligt præcis hvilket er eftertragtet.

Generelle antagelser om vægt er lavet ud fra et simpelt design og en antaget vægt på omkring 100g

pr. motor, 700g løftevægt (som kriterie i grundspecifikation), og antaget vægt af strukturelle dele for armen. Disse udregninger ses i Figur 6 og i komplet form i Appendix E.

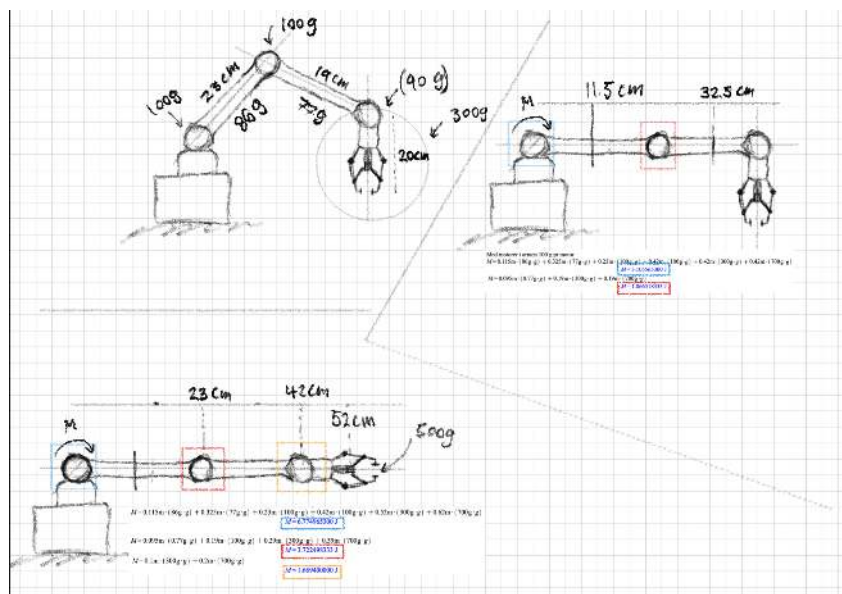


Figure 6: Styrkeberegninger

Som set ville det maksimalt krævede moment være omkring $6.7 \text{ N}\cdot\text{m}$. De fleste prisvenlige steppermotorer vil oftest levere omkring $3\text{-}4 \text{ N}\cdot\text{m}$, selv ved gearing...

Stepper motor konfiguration kasseres, og der udvikles herfra på servomotor konfigurationen (BLÅ).

3.5 Designbeslutninger og overgang til konstruktion

Efter udvælgelsen af de tre koncepter gennem det morfologiske skema og de tilhørende styrkeberegninger, blev det besluttet at basere den endelige konstruktion på **servomotor-konfigurationen (blå koncept)**. Denne beslutning blev truffet på baggrund af en række tekniske og praktiske overvejelser, som skabte grundlaget for den videre konstruktion.

For det første viste momentberegningerne, at prisvenlige steppermotorer ikke kunne levere tilstrækkeligt drejningsmoment til at opfylde de krav, der var defineret i grundspecifikationen, særligt i forhold til løft af op til 700 gram og bevægelse i flere frihedsgrader. Af den grund blev steppermotor-konfigurationen (rød) fravalgt.

Servomotorer blev vurderet som en mere egnet løsning, da de, udover at kunne levere højere moment, også tillader enklere ledstyring via PWM og dermed muliggør en mere direkte integration med Arduino og det øvrige styresystem. Derudover understøttede servokonfigurationen et mere kompakt og modulært design, hvilket stemte overens med ønsket om høj brugervenlighed og lav vedligeholdelse.

På baggrund af ovenstående blev der defineret en række **centrale designprincipper** for overgang til konstruktionsfasen:

- **Valg af motorer:** Hver servomotor blev valgt med udgangspunkt i momentkravene for det enkelte led – kraftige motorer til skulder og albue, og mindre motorer til håndled og griber.

- **Produktionsmetode:** Alle komponenter skulle kunne 3D-printes med hensyn til styrke, præcision og tolerancer. Det gav mulighed for hurtig iterativ udvikling.
- **Strøm og kommunikation:** En decentral og trådløs opsætning med ESP8266, Raspberry Pi og Arduino blev valgt for at sikre fleksibilitet og skalérbarhed.
- **Sensorisk feedback:** En taktil sensor blev integreret i robtohånden for at kunne justere grebssstyrken baseret på objektets hårdhed – et vigtigt skridt i retning af adaptiv og sikker håndtering.

Med disse overvejelser som baggrund blev projektet ført videre til konstruktionsfasen, hvor de teoretiske koncepter blev omsat til fysiske prototyper og et fuldt fungerende mekatronisk system.

4 Konstruktionsfase

Efter udvælgelsen af det endelige koncept og analysen af armens nødvendige bevægelsesfrihed, blev projektet ført videre til konstruktionsfasen. I denne del af arbejdet blev de teoretiske overvejelser og designvalg omsat til fysiske prototyper og tekniske løsninger. Fokus lå her på den konkrete opbygning af robotarmen, fra 3D-design og materialevalg til elektrisk opsætning, motorvalg og programmering. Samtidig dokumenteres vigtige hensyn som tolerancer, strømforsyning og integration af software og hardware.

4.1 Overordnet opsætning

Konstruktionen består af en kvadratisk træplade (1×1 m), hvorpå robotarmen og tilhørende komponenter er fastmonteret. Robotarmen er centralt placeret ved pladens bundkant og består af en 3D-printet, cirkulær base. Basen er hul og rummer både en Arduino, en ESP8266, et breadboard, samlemuffer og en servomotor til rotationsbevægelse af basen. Øvrige led i robotarmen er ligeledes 3D-printede og indeholder hver sin servomotor, som styrer armens bevægelser i sine frihedsgrader.

Til højre for robotarmen er en strømforsyning fra en stationær PC monteret fast. Umiddelbart ved siden af er tre buck converters opstillet i et 3D-printet tårn, som fordeler korrekt spænding til de forskellige servomotorer.

Over arbejdsområdet er et stativ opstillet i form af et lodret kasteskaf, hvorpå en lille platform er monteret. Her er en Raspberry Pi med tilhørende Raspberry Pi Camera Module 3 fastgjort. Kameraet er vinklet ned mod arbejdsfladen for at sikre fuldt visuelt overblik. Platformen er desuden udstyret med et vaterpas for at sikre korrekt vandret positionering ved indstilling.

Arbejdsområdet er defineret som det område på pladen, der befinder sig til højre for armens rotationscentrum og under den fastsatte gule horisontale reference linje. Begrænsningerne på arbejdsområdet er fastsat for at forhindre, at kameraet registrerer robotarmen, strømforsyningen, sorteringskasserne og

lignende komponenter. Kameraet anvendes til at lokalisere og identificere objekter ved hjælp af billedbehandling og AI. Robotarmen er designet til herefter at kunne interagere med objekterne via en robtohånd, som er udstyret med en taktil sensor.

4.2 CAD og Prototypedesign

I projektet er der blevet designet og udviklet to prototyper af en robotarm samt én prototype af en robot-hånd. Hele udviklingen blev lavet i Solidworks, hvor der blev lagt særligt fokus på tolerancer, 3D-print design og funktionalitet i forhold til mekanik og motorvalg.

4.2.1 Prototype 1 – Steppermotor

I den første prototype blev der brugt en steppermotor til rotationen i basen. For at overføre drejningsmomentet blev der lavet et slot-fit design mellem motorens tandhjul og det tandhjul, der sad på akslen. Dette sikrede, at remmen mellem tandhjulene altid var i spænd. Dog viste det sig, at steppermotoren ikke kunne levere tilstrækkeligt moment i de forskellige led, især skulderleddet, hvilket førte til udviklingen af prototype 2.

4.2.2 Prototype 2 – Servomotor

Efter skiftet til en servomotor i prototype 2 blev det nødvendigt at redesigne hele armen, da servoens mål og rotationsbegrænsninger var anderledes. Designet blev gjort mere kompakt, og servoen blev monteret direkte i basen, hvilket gjorde det muligt at forenkle designet. Begge prototyper kan ses i Appendix C.

4.2.3 3D-print og tolerancer

Alle plastdele blev 3D-printet på en Prusa MK3.5S med PLA. Generelt blev der anvendt 25 % infill til de fleste komponenter, da dette gav en god balance mellem styrke og printtid. For mere udsatte komponenter, såsom *fingerspidserne i robot-hånden*, som også fungerer som snap-fit forbindelser, blev infillen øget til 100 % for at undgå brud under montering og brug.

Ved design af snap-fit forbindelser, hvor en kugle skal klikkes ned i et hul, blev tolerancen typisk sat til omkring +0.2 til +0.3 mm på kuglen i forhold til hullets indre diameter. Dette skabte den nødvendige spænding, samtidig med at det stadig var muligt klikke fingerspidsen fast.

I tilfælde hvor motorer eller lejer skulle kunne monteres og afmonteres uden værktøj, men stadig sidde med god pasning, blev tolerancerne sat til cirka 0.25 mm frigang. Til frie rotationer – eksempelvis rotationsemnet i basen blev tolerancen øget til 0.5–0.75 mm, hvilket sikrede, at der ikke opstod uønsket friktion efter print.

4.3 Beskrivelse af det mekatroniske setup

Det mekatroniske setup består af et modulært og trådløst styret system, der samlet koordinerer robotarmens bevægelser. Øverst i opsætningen er et Raspberry Pi Camera Module 3 monteret på et separat

stativ, uafhængigt af selve robotarmen. Kameraet er forbundet til en Raspberry Pi, som fungerer som systemets centrale billedbehandlings- og kommunikationsenhed. Fra Raspberry Pien sendes styringssignaler trådløst via WiFi til en ESP8266 placeret i robotarmens base.

ESP8266'en fungerer som mellemed og sender signalerne videre via kabel til en Arduino, som agerer som slaveenhed og står for den direkte kontrol af de fem servomotorer, der driver robotarmens bevægelige led.

Til strømforsyning anvendes en modificeret strømforsyning fra en stationær PC. En taktil sensor er monteret på fingerspidsen af robothånden. Denne måler kontaktryk og anvendes til at sikre, at robotarmen tilpasser grebsstyrken efter objektets hårdhed, så både bløde og hårde genstande kan håndteres skånsomt og effektivt.

Samlet set muliggør opsætningen en fleksibel og responsiv robotarm, hvor billedbehandling og kraftfeedback integreres i et trådløst mekatronisk system med decentral styring. Nedenfor i Figur 7 ses et simplificeret diagram over det mekatroniske setup

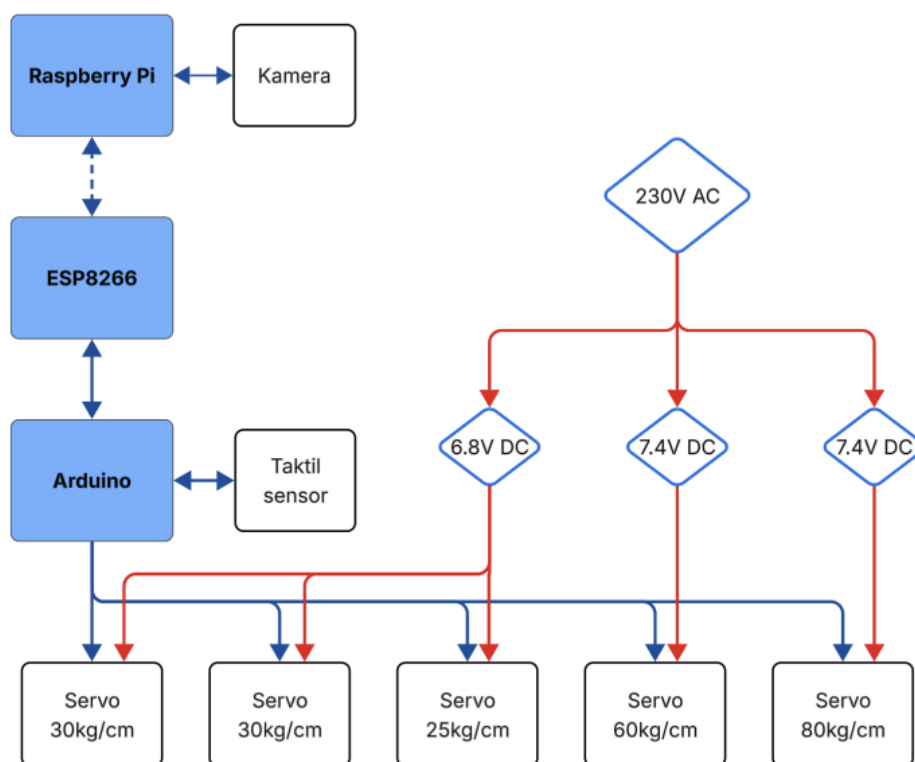


Figure 7: Simplificeret mekatronisk setup

4.3.1 Strømforsyning

De forskellige motorer stillede forskellige krav til strømspændingen for at arbejde ved optimal styrke. Det medførte også at der skulle findes en prisvenlig og modulær løsning til leveringen af separate strømoutputs fra et fælles 220V input.

Efter udforskning af muligheder blev en 305W ATX Power Supply fra en gammel DELL Stationær Computer udvalgt. Den overholdte kriterier for høj Ampere og mulighed for levering af forskellige spændinger, inklusiv 12V, samtidigt med at være sin egen enkelte enhed.

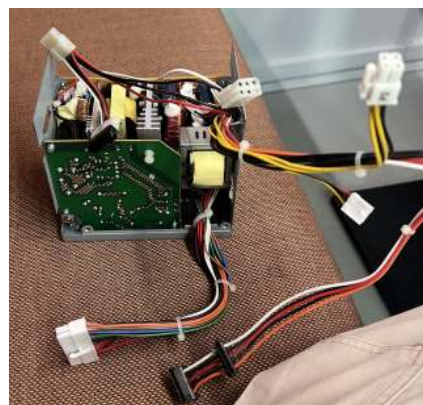


Figure 8: Strømforsyning før modificering

Modifikationer var krævet for at strømforsyningen kunne levere den fornødne strøm samt overkomme de restriktioner og sikkerhedsforanstaltninger der var indbygget for at forhindre overløb og kortslutning fra fabrikken.

Tre 12V-udgange herfra føres gennem tre buck converters, som nedtransformerer spændingen til henholdsvis 7,4V, 7,4V og 6,8V. De to 7,4V-udgange forsyner hver sin store og kraftige servomotor, mens 6,8V-udgangen deles mellem tre mindre servomotorer med lavere strømkrav. Alle GND-forbindelser fra servomotorer og strømforsyning samles og forbindes til Arduinoens fælles jord.

4.3.2 Motorspecifikationer

Robotarmen er opbygget med fem servomotorer, der hver især er valgt ud fra momentkravene i deres specifikke led. Skulderleddet drives af en kraftig 80 kg/cm servo med et moment på ca. 7,85 Nm, da dette led skal bære størstedelen af armens vægt. Albueleddet er udstyret med en 60 kg/cm servo (ca. 5,88 Nm), som også håndterer betydelig belastning. Håndleddet og gribefunktionen betjenes af to mindre servoer med et moment på 30 kg/cm (ca. 2,94 Nm), som er tilstrækkeligt til deres arbejdsbyrde. Til sidst anvendes en 25 kg/cm servo (ca. 2,45 Nm) til rotation af basen, hvor momentbehovet er lavere. Alle servoerne har en rotationsvinkel på 180 grader, hvilket giver god bevægelsesfrihed i hele armen. For servo-specifikationer se Appendix A.

4.4 Software

4.4.1 Kamera og billedbehandling

Til bestemmelse af objektets position anvendes et **Raspberry Pi Camera Module 3**, som er monteret på et stativ over robotarmen, så det har fuldt overblik over arbejdsområdet. For at kameraet kan omsætte pixels til faktiske afstande er der placeret to ArUco-markører i hjørnerne af pladen. ArUco-markører er sort-hvide kvadratiske koder med et unikt binært mønster, som gør det muligt entydigt at identificere

dem og bruge dem som referencepunkter til skalering og orientering.[7]

Til kalibrering af kameraets linse anvendes en *chessboard calibration*-markør, som sikrer korrekt afstandsberegning uanset vinkel, forvrængning eller linseforhold (fx ved fisheye-effekt).

Efter billedet er taget, bruges **OpenCV** i Python til at:

1. Identificere ArUco-markørernes placering
2. Croppe billedet til det relevante arbejdsområde
3. Bestemme centrum af robotarmen ud fra data fra SolidWorks
4. Tegne en linje, som adskiller robotarm, elektronik og sorteringsområde fra resten af billedet og dermed definere et fokusområde

Herefter konverteres billedet til sort/hvidt, og **OpenCV**'s **contour**-funktion bruges til at identificere det største objekt i fokusområdet(appendix G). Objektet afgrænses med en bounding-boks, og midten af boksen findes. Fra robotarmens centrum til objektets midte dannes en vektor i pixels. Ved hjælp af ArUco-markørerne konverteres denne vektor til meter. Se appendix H, for annoteret billede.

4.4.2 Samarbejde med OpenAI

Når objektet er lokaliseret, croppes det ud igen og gemmes som et nyt billede. Dette billede sendes sammen med en tekstprompt til OpenAI's API via Python. OpenAI analyserer billedet og returnerer:

- Objektets type
- Estimeret vægt
- Objektets højde
- Anbefalet tryk for at kunne løfte det uden at beskadige det
- Ewaste eller ikke Ewaste

Disse data benyttes til at justere tryksensorernes grænser i robotens gribemekanisme, så hånden automatisk tilpasser sig det enkelte objekts egenskaber. Se tekst-prompt i appendix F.

4.4.3 URDF-fil

I forbindelse med udviklingen af robotarmen er der udarbejdet en URDF-fil (Unified Robot Description Format) gennem en udvidelse i SolidWorks. URDF-filen har til formål at beskrive robotarmens geometri, led og fysiske egenskaber, så software kan forstå hvordan robotten hænger sammen. Dermed danner URDF-filen grundlag for simulering, kinematikberegninger og softwarebaseret styring af robotten.

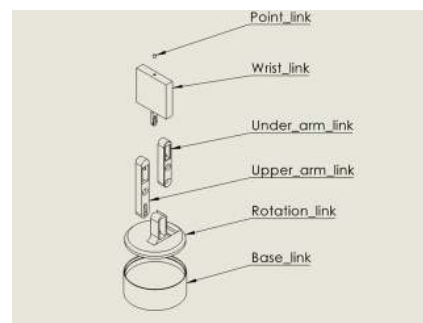


Figure 9: Hver del og navn i software

URDF-filen er skabt ved hjælp af en udvidelsespakke til SolidWorks, som er downloadet fra ROS' egen documentation side [8]. Udvidelsespakken gør det muligt at generere URDF-filer direkte fra en assembly-fil i SolidWorks. Gennem `export_to_URDF`-funktionen vælges først et base-link, i dette tilfælde vores bund, og efterfølgende tilhørende links - se Figur 9. Når alle links er valgt, skal der angives, hvordan hvert led bevæger sig. I robotarmens tilfælde er alle led defineret som revolute joints, hvilket betyder, at de tillader rotation omkring én akse. Revolute joint er et mekanisk led, som tillader en relativ rotation mellem to sammensatte komponenter omkring en fælles akse. Efter komponenter og led er defineret sættes begrænsninger, hastigheder og kræfter for de forskellige led, hvorefter URDF-filen kan blive eksporteret.[9]

Gennem arbejdet med URDF-filen opstod flere problematikker, som er vigtige at være opmærksom på når man udarbejder sin fil gennem SolidWorks.

Koordinatsystem og akser: For hvert led er det afgørende, at det tilhørende koordinatsystem og akser er hensigtsmæssigt defineret i forhold til geometrien, placeringen og leddets funktion. SolidWorks tilbyder en funktion til automatisk generering af disse, men denne bør anvendes med forsigtighed, da resultatet er følsomt over for komponentens placering i assemblyen. Dette kan medføre, at akserne placeres uhensigtsmæssigt i forhold til den ønskede bevægelse. Det anbefales derfor enten at definere koordinatsystem og akser manuelt ved hjælp af referencegeometri eller nøje at gennemgå og eventuelt justere de automatisk genererede.

Closed loop: Den originale hånd fungerede som en closed loop gribemekanisme, hvilket vil sige at flere links gensidigt påvirker hinanden via flere forbindelser. Dette kræver altså kinematiske kredsløb, som ikke understøttes af URDF. Denne problematik blev løst ved at vælge at se bort fra håndens funktion i URDF-filen. Ved at udskifte selve hånden i assembly-filen, med en klods af samme geometri, var det muligt at lave en URDF-fil, som beskriver rotation, `upper_arm`, `lower_arm` og `wrist`-komponent9. Derved er selve armens geometri og funktion beskrevet korrekt, derved vil software kunne bearbejde det. [10]



Figure 10: Preview af URDF-filen af armen

4.4.4 Invers kinematics

For at kunne bevæge robotarmens endeeffektor (spidsen) hen til en ønsket position i rummet, er det nødvendigt at kende de præcise vinkler for hver af armens fem led. Denne proces kaldes inverse kinematik og udgør det modsatte af den fremadgående kinematik, hvor man i stedet udregner endeeffektors position ud fra kendte ledvinkler.[11]

Robotarmen er opbygget af fem sammenkædede led, og bevægelsen i hvert led kan beskrives matematisk gennem transformationer. Disse beskriver, hvordan hvert led bidrager til armens samlede position og orientering i rummet. Til dette anvendes Denavit-Hartenberg-parametre, som giver mulighed for at opstille en samlet transformationsmatrix:

$$T_{\text{total}} = T_1 \cdot T_2 \cdot T_3 \cdot T_4 \cdot T_5$$

Her repræsenterer hver T_i den homogene transformationsmatrix for led i . Målet med inverse kinematik er at finde de vinkler $\theta_1, \theta_2, \theta_3, \theta_4, \theta_5$, som opfylder:

$$T(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5) = T_{\text{ønsket}}$$

hvor $T_{\text{ønsket}}$ er den ønskede position og orientering af robotarmens spids. Hver matrix T_i har følgende form:

$$T_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Her indgår DH-parametrene:

- θ_i : rotationsvinklen for led i

- a_i, d_i, α_i : geometriske parametre som bestemmer længde og vinkel mellem led

Ved at multiplicere alle transformationerne opnås en samlet matrix, som indeholder information om både position (x, y, z) og orientering. Problemet reduceres herefter til at finde de θ_i -værdier, som løser ligningssystemet[12]:

$$f(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5) = \begin{bmatrix} x_{\text{ønsket}} \\ y_{\text{ønsket}} \\ z_{\text{ønsket}} \\ \text{orientering} \end{bmatrix}$$

Da denne type ligningssystem sjældent har en analytisk løsning, anvendes typisk numeriske metoder som eksempelvis Newton-Raphson eller optimerings algoritmer.[13] I projektet er beregningen af inverse kinematik løst vha. Python-biblioteket IKPy.

IKPy gør det muligt at finde en numerisk løsning af inverse kinematik-beregninger baseret på den udarbejdede URDF-fil. Grundlæggende fungerer IKPy ved først at indlæse robotbeskrivelsen gennem URDF-filen i form af længder mellem led, ledtyper, orienteringer af hver led og fysiske begrænsninger. IKPy fortolker URDF'en som en kæde af led og links, hvor hvert led har sin egen transformationsmatrix. Hele robotten beskrives altså matematisk som en kæde af homogene transformationsmatricer. Givet et input i form af en position og eventuel orientering forsøger IKPy at finde de ledvinkler, som bringer robotarmens sidste led så tæt på som muligt. Af denne årsag var det derfor nødvendigt at tilføje et yderligere led i spidsen af armen, så vi på den måde fik håndens geometri med i beregningen.

Under udviklingen fungerede inverse kinematik-beregningerne ikke alene som et værktøj til styring, men også som en metode til validering af URDF-filen. Eventuelle fejl i URDF'ens definition af led, akselretninger eller koordinatsystemer blev identificeret gennem afvigelser i de beregnede ledvinkler eller urealistiske positioneringsresultater. På den måde kunne man altså sikre sig en sammenhæng mellem den fysiske robotarm og den digitale model anvendt i beregningerne. IKPy tager også højde for vinkelbegrænsninger fra URDF'en, så løsningen kun foreslår ledvinkler, der passer indenfor robotarmens operationsområde.

Disse begrænsninger kan dog give store problemer for IKPy. Når IKPy løser inverse kinematik, gør den det numerisk, hvilket grundlæggende vil sige at algoritmen ikke beregner en direkte løsning via matematiske formler, men i stedet forsøger sig frem ved at afprøve og gradvist forbedre ledvinklerne for at nærme sig den ønskede position. Frem for en analytisk algoritme benytter IKPy altså en iterativ søgealgoritme. Det vil sige at den starter i et punkt, udregner hvor tæt denne løsning er på målet via en fejlfunktion eksempelvis Euklidisk afstand, og derefter justerer vinklerne lidt ad gangen for at mindske fejlen. Denne proces gentages indtil tilfredsstillende løsning er fundet eller algoritmen opgiver. Grundet dette kan begrænsninger i URDF'en give problemer, da algoritmen kun må afsøge løsninger inden for de tilladte vinkelområder. Hvis ledbegrænsning blokerer en ellers oplagt retning i søgningen kan opti-

meringen køre fast. Særligt kan dette ske hvis flere led er afhængige af hinanden for at opnå en position, hvilket stort set altid vil være tilfældet for robotarmen. [14]

Denne problematik viste sig i arbejdet med robotarmen, da servomotorerne kun kunne bevæge sig 180 grader blev disse begrænsninger indsat i URDF-filen. Dette medførte dog låste led i inverse beregningerne, og gav derfor store fejl. `rotation_joint9` havde fået sat en begrænsning på $[0^\circ, 180^\circ]$ og `upper_arm_joint9` $[0^\circ, 90^\circ]$. Målpositionen kan godt nås fysisk, men kun hvis `rotation_joint9` står på fx 90 grader, så `upper_arm_joint` kan "bøje fremad" inden for sin begrænsning. Hvis IKPy under optimering prøver sig frem og fx starter med (`rotation_joint = 180^\circ`), vil den opdage at `upper_arm_joint` ikke længere kan bøje bagover for at kompensere, fordi det kræver en vinkel udenfor begrænsningen. Dermed sidder algoritmen fast i et hjørne af søgeområdet, hvor den ikke kan komme tættere på målet. IKPy har altså ikke nogen global forståelse.

For at løse problemet, blev det besluttet at fjerne begrænsningerne i URDF-filen, og i stedet løse dem gennem kodning. Ved at give IKPy nogle bud på startværdier, kan man ved kontrol finde en gyldig løsning, som ikke overskrider begrænsningerne.

4.4.5 Kompensation for kalibrering i servomotorer

Under arbejdet med servomotorerne blev det hurtigt tydeligt, at de ikke kommunikerede korrekt med Arduino-koden. Servomotorerne har et nominelt bevægelsesområde mellem 0 og 180 grader, men når værdier som eksempelvis 90 eller 180 grader blev sendt, afveg motorernes faktiske positioner betydeligt fra de forventede. Hver motor havde individuelle forskelle i både minimums- og maksimumsvinkler, og deres vinkelrespons var desuden skaleret forskelligt.

For at kompensere for disse afvigelser blev hver enkelt servomotor kalibreret ved måling i flere referencepositioner. På baggrund af disse målinger blev der opstillet en kompenserende lineær funktion, som kunne omregne de beregnede inverse kinematik-vinkler til de korrekte servo-kommandoer. Herved blev det muligt at sikre, at de vinkler, som blev beregnet i IKPy-softwaren, kunne omsættes korrekt til servomotorernes fysiske positioner via Arduino-styringen og dermed resultere i den ønskede bevægelse af robotarmen.

Eksempelvis omregnes den givne vinkel fra IKPy til `wrist_joint`:

$$\theta_{Arduino} = 159 - |\text{round}(\theta_{IKPy} \cdot 0.7278 + 14)|$$

4.4.6 Overordnet system

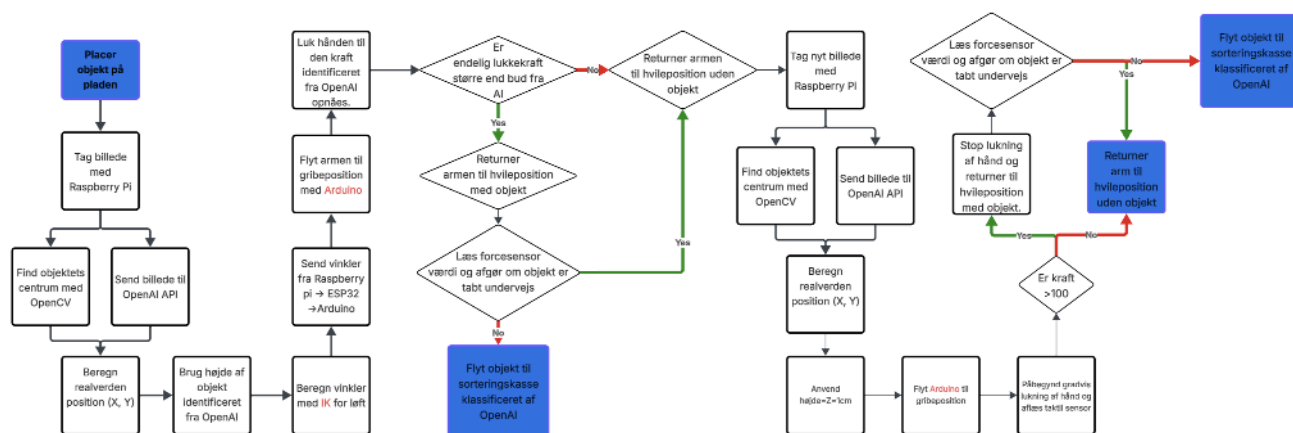


Figure 11: Flowchart over ønsket proces

Det tilstræbes, at proceduren opdeles i to overordnede faser. I den første fase, DoAi, ønskes det, at kameraet i samspil med OpenAI's API anvendes til at estimere objektets højde, nødvendige løftekraft samt sorteringskategori. På baggrund af disse data skal robotarmen forsøge at løfte og sortere objektet uden yderligere sensorisk feedback. Dog anvendes kraftsensoren til at vurdere om objektet er løftet eller tabt i forsøget.

Hvis dette løftforsøg ikke lykkes, ønskes en anden fase aktiveret, nemlig DoHuman, hvor systemet anvender indbyggede sensorer, herunder taktile kraftsensorer. Denne fase kaldes DoHuman da den imiterer mennesket, i det at den tager stimuli fra omgivelserne i forsøget på at løfte, objektet. I denne fase skal kameraet fortsat identificere objektet og dets sorteringskategori, hvorefter griberen gradvist lukkes om objektet, indtil en foruddefineret kraftgrænse er nået. Når denne threshold er opnået, skal objektet løftes og sorteres.

Formålet med denne tilgang er at opnå et robust samspil mellem AI-baseret billedanalyse og fysiske sensorer. Den kombinerer en moderne metode til intelligent sortering baseret på kunstig intelligens med en fallback-mekanisme i form af mekanisk feedback, hvilket forventes at øge systemets pålidelighed ved fejl eller usikkerhed i AI'ens estimater. Et flowchart over processen er visualiseret i Figur 11. Den færdigt udarbejdede konstruktion monteret og klar til brug kan ses på Figur 12.



Figure 12: Færdig udarbejdet robotarm

Af 11 fremgår det, at objektet sorteres som enten EWaste eller ikke-EWaste, baseret på AI'ens vurdering. Kategorierne er fleksible og kan udvides eller tilpasses efter behov.

5 Test af systemets funktionelle egenskaber

5.1 Positionsnøjagtighed

For at undersøge robotarmens præcision ved gribebevægelser blev en rektangulær testklods gentagne gange placeret i forskellige positioner i arbejdsområdet. Formålet var at vurdere, hvor symmetrisk armen griber om objektet i forhold til dets centerlinje. I hver gentagelse blev afstanden fra klodsens midtpunkt til venstre og højre gribearm målt manuelt.

Målingen blev udført 15 gange, med forskellige startpositioner, men med samme kommando til gribebevægelsen. Forskellen mellem de to afstande er et mål for forskydningen, og dermed et mål for præcisionen i gribefasens centrering. Forskydningen beregnes som:

$$\delta_i = a_{\text{højre}}^{(i)} - a_{\text{venstre}}^{(i)}$$

hvor $a_{\text{højre}}^{(i)}$ og $a_{\text{venstre}}^{(i)}$ er afstanden fra klodsens midte til henholdsvis højre og venstre gribearm i måling i . Forskydningsretningen er defineret som positiv, hvis klodsens midte er forskudt mod venstre, og negativ, hvis den er forskudt mod højre. Et lille udtag af resultaterne fra forsøget kan ses i Table 1. De fulde forsøgsresultater kan findes i Apendix B.

Måling	a_{venstre} [mm]	$a_{\text{højre}}$ [mm]	δ [mm]	Retning
1	40	60	20	Venstre
2	60	40	-20	Højre
3	65	35	-30	Højre
4	55	45	-10	Højre

Table 1: Udkast fra målinger af afstanden fra klodsens centrum til hver gribearm, og beregnet forskydning.

Resultaterne kan opsummeres som følger:

- Antal målinger: $N = 15$
- Gennemsnitlig forskydning: $\bar{\delta} = 9.33$ mm
- Standardafvigelse: $\sigma = 21.87$ mm
- Maksimal absolut forskydning: $|\delta|_{\text{maks}} = 40$ mm

5.2 Gentagelsesnøjagtighed

Udover positionsnøjagtighed ønskes det at undersøge robotarmens evne til at udføre gentagne bevægelser med høj præcision. Dette er særligt relevant, da armens tiltænkte funktion er at udføre sorteringsopgaver, hvor ensartet placering er essentiel for et pålideligt resultat.

Testen blev udført ved, at robotarmen gentagne gange løftede et objekt og placerede det i en foruddefineret position baseret på faste koordinater målt af kameraret. Efter hver placering blev objektets position igen målt med kameraret og forsøget gentaget 10 gange under identiske betingelser, så det forrige punkt fungerede som referencepunkt. Målet med forsøget er at identificere potentielle variationer i placeringen over tid, som kan skyldes f.eks. mekanisk slør i leddene, variation i motorrespons eller ophobning af små unøjagtigheder.

Afviselserne i gentagelserne opstilles ved at beregne den gennemsnitlige afstand fra det ønskede referencepunkt til hvert observeret punkt:

$$\delta_i = \sqrt{(x_i - x_0)^2 + (y_i - y_0)^2}$$

hvor (x_0, y_0) er de ønskede koordinater, og (x_i, y_i) er målt placering ved gentagelse i . Et lille udtag af forsøgsresultaterne kan ses i Table 2. De fulde forsøgsresultater kan findes i Appendix B.

Måling	x_i [mm]	y_i [mm]	δ [mm]
1	34.4	28.9	1.94
2	34.2	29.9	1.02
3	33.9	30.1	0.36
4	34.0	30.1	0.10

Table 2: Afstand fra hvert punkt til referencepunktet: Måling 1 til (35.5, 27.3), og herefter hver måling relativt til den foregående.

Ud fra de målte afvigelser udregnes følgende:

- Antal gentagelser: $N = 15$
- Gennemsnitlig afvigelse: $\bar{\delta} = 0.43$ mm
- Standardafvigelse: $\sigma = 0.60$ mm
- Maksimal afvigelse: $\delta_{\text{maks}} = 1.94$ mm

Testen giver et billede af, hvor konsekvent robotarmen kan placere et objekt ved gentagne udførsler, og afslører eventuelle systematiske fejl eller slør i bevægelserne.

5.3 Test af løfteevne

Formålet med dette forsøg er at bestemme robotarmens maksimale løftekapacitet samt identificere eventuelle mekaniske begrænsninger. Forsøget er relevant, da løfteevne er en afgørende faktor i mange sammenhænge med sortering, og kritisk for en bredere portfolio af arbejdsopgaver.

Forsøget blev udført ved at placere en plastikflaske i en fast defineret position, hvorefter robotarmen løftede flasken til en forudbestemt højde. Efter hver vellykket løftesekvens blev vand hældt i flasken, for at øge vægten. Processen blev gentaget indtil robotarmen ikke længere kunne løfte flasken. Et fejlet løft er defineret som en af følgende overtrædelser:

- mekanisk svigt (f.eks. tab af objekt under løft),
- overskridelse af servoens momentgrænse,
- betydelig nedsættelse af hastighed eller præcision.

Ved hver gentagelse blev vægten af flasken noteret, og forsøgets resultater blev anvendt til at vurdere, hvor i kæden den største svaghed opstod f.eks. ved et specifikt led eller motor. Forsøgsresultaterne kan findes i Table 3.

Forsøg	Total vægt [g]	Løft succesfuld (Ja/Nej)
1	250	succes
2	326	succes
3	437	succes
4	502	fejlet

Table 3: Resultater fra løfteevne-test med gradvist øget belastning.

- Maksimalt velløftet masse: $m_{\text{maks}} = 437$ g
- Antal løft: $N = 4$
- Begrænsende komponent identificeret: Servomotor i håndled.

Resultaterne giver ikke blot et mål for armens løfteevne, men også et indblik for videreudvikling. Eksempelvis kan forstærkning af specifikke led eller udskiftning til motorer med højere drejningsmoment være nødvendige for at forbedre ydeevnen.

5.4 Test af AI-klassifikation

Formålet med dette forsøg er at evaluere nøjagtigheden af det anvendte klassifikationssystem, som anvender OpenAI's billedbehandlingsmodel til at identificere og kategorisere affaldsobjekter som enten *E-Waste* eller *Ikke E-Waste*. Systemets klassifikation er essentiel for processen da sorteringen af skraldet sker på baggrund af den.

Til forsøget blev der udvalgt 15 forskellige objekter med kendt kategori, heraf 8 som entydigt kan betegnes som *E-Waste* og 7 som *Ikke E-Waste*. Hvert objekt blev enkeltvis placeret i arbejdsområdet under kameraet, hvorefter et billede blev taget og sendt til OpenAI's billedgenkendelsesmodel via API. Et lille udtag af resultaterne fra forsøget kan ses i Table 4. Resten af resultaterne kan findes i Appendix B.

Faktiske Objekt	Klassificeret objekt	Faktisk kategori	Klassificeret som	Korrekt (Ja/Nej)
PS4-controller	Controller	E-Waste	E-Waste	Ja
Mus	Mus	E-Waste	E-Waste	Ja
Kuglepen	Kuglepen	Ikke E-Waste	Ikke E-Waste	Ja
Tomt iPhone cover	Telefon cover	Ikke E-Waste	Ikke E-Waste	Ja

Table 4: Forsøgsresultater fra klassifikationstest med navngivne objekter.

Med resultaterne fra forsøget kan en Konfusionsmatrix opstilles. Her inddeles resultaterne fra forsøget i True positive(TP), False negative(FN), False positive(FP) og True negative(TN). Konfusionsmatricen kan ses i Table 5.

	Klassificeret som E-Waste	Klassificeret som Ikke E-Waste
Faktisk E-Waste	8 (TP)	0 (FN)
Faktisk Ikke E-Waste	2 (FP)	5 (TN)

Table 5: Konfusionsmatrix for klassifikationstest

Ud fra konfusionsmatricen kan klassifikationssystemets accuracy (nøjagtighed) og precision (præcision) nu beregnes. Accuracy angiver, hvor stor en andel af alle klassifikationer der er korrekte, mens precision specifikt viser, hvor stor en andel af de objekter, der klassificeres som E-Waste, der faktisk er E-Waste.

Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{8 + 5}{8 + 5 + 2 + 0} = \frac{13}{15} \approx 0,867$$

Modellen klassificerede altså cirka 86,7% af alle objekter korrekt.

Precision:

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{8}{8 + 2} = \frac{8}{10} = 0,8$$

Dette betyder, at når modellen klassificerede et objekt som *E-Waste*, var den korrekt i 80 % af tilfældene.

6 Videreudviklingsmuligheder

Robotarmen har stort potentiale for videreudvikling, både hvad angår hardware, software og kunstig intelligens. En oplagt forbedring vil være at udskifte de anvendte servomotorer med mere præcise og kontinuerligt roterende motorer. Dette ville ikke blot reducere afrundingsfejl ved konvertering af in-verse kinematiske løsninger til Arduino-kompatibel kode, men også muliggøre en forbedret løftekapacitet afhængigt af den valgte motor.

Størstedelen af robotarmens komponenter er fremstillet ved hjælp af 3D-print. For at øge robustheden og reducere mekanisk slør i leddene, kan man med fordel implementere mere præcise samlingsmetoder såsom gevindindsatser, kuglelejer samt bolte og møtrikker. Dette vil forbedre både nøjagtighed og holdbarhed i længerevarende brug.

Ledningsføringen (cable management) kan optimeres yderligere, eksempelvis ved at farvekode kablerne efter funktion og komponent samt samle dem i kabelstrømper eller slanger for et mere overskueligt design.

Den nuværende gribemekanisme er konstrueret som en lukket kinematisk kæde (closed loop), hvilket ikke understøttes i URDF-formatet. En fremtidig opgradering kunne bestå i at redesigne griberen som en åben kinematisk kæde. Dette vil tillade præcis simulering og modellering af griberen i URDF-baseret simulering og samtidig eliminere behovet for manuel kalibrering af gribeafstand under test.

På software- og AI-siden kan robotarmens funktionalitet specialiseres væsentligt gennem udvikling af egne billedgenkendelsesmodeller. Dette kan forbedre pålideligheden ved identifikation af specifikke komponenter, som robotten forventes at håndtere. Alternativt vil man i takt med udviklingen af større og mere robuste billedgenkendelsesmodeller, eksempelvis fra OpenAI, kunne drage fordel af den forbedrede generaliseringsevne over tid.

7 Diskussion

De udførte tests viser, at robotarmen opnår en rimelig høj præcision og gentagelsesnøjagtighed under kontrollerede forhold. Den gennemsnitlige forskydning ved gentagelse lå på $\bar{\delta} = 0.43$ mm med en standardafvigelse på $\sigma = 0.60$ mm, hvilket indikerer, at armen er i stand til at gentage sine bevægelser med god nøjagtighed.

Dog er der en række usikkerheder og begrænsninger, der bør diskuteres. For det første er armens mekanik primært baseret på 3D-printede komponenter og billige servomotorer, hvilket medfører slør og positionsusikkerhed, særligt ved høj belastning. Dette sås tydeligt i løftetesten, hvor et enkelt led mistede evnen til at løfte, allerede ved relativt lav masse. Samtidig afslører forskydningen i grebsmidte (op til 40 mm) en vis usikkerhed i armens gribemekanisme og kalibrering. Disse skyldes i største omfang

oversættelsen fra inverse kinematik værdier til Arduino kommandoer. Her har det været nødvendigt at opstille funktioner, til omregning, som er lavet på få punkter vurderet ved øjemål. Ved at tilføje bedre servomotorer ville man kunne undgå dette. Ved at vælge kontinuerede servomotorer ville man samtidig kunne skabe en bedre mobilitet af armen, hvilket ville give muligheder for mere komplekse opgaver.

Desuden kan URDF-modellen være begrænsende. URDF understøtter ikke lukkede kinematiske kæder, hvilket gør det svært at modellere en realistisk gribemekanisme. Dette kræver i praksis manuel kalibrering af gribeafstand og kompenserende tilpasninger i softwaren. Dette har yderligere være med til at skabe usikkerheder, og kan give problematikker ved komplekse former, da softwaren ikke er i stand til at simulere gribemekanismens kontakt med objektet. Dette kan løses ved brug af en gribemekanisme baseret på åbne kinematiske kæder.

Sensorisk feedback har vist sig essential i form af kraftsensorer. Under forløbet er to servomotorer i håndens lukkemekanisme brændt sammen grundet for højt tryk under løft. Efter indførslen af kraftsensorer har håndens servomotor ikke vist bekymrende temperaturer ved brug af AI. Dette er dog ikke tilfældet når robotten fejler ved AI og overgår til "Do Human"-programmet. Her vil hånden overophede over længere løftesekvenser.

Det har været forsøgt at tilføje en laser afstandsmåler, på hånden for at kunne måle højden af de objekter, som skulle løftes. Dette har dog vist sig at være mere indviklet end først forventet. Grundet IKPy's numeriske løsningsalgoritme, har begrænsninger været til stor problematik, og løst ved at give IKPy "initial guesses" og begrænsninger i selve inverse kinematik koden. For at kunne benytte laseren har det været nødvendigt at låse `wrist_link` i vandret plan, men denne yderligere begrænsning låser armen, eller overskider de andre begrænsninger. Det antages dog at være muligt.

Endeligt er AI-genkendelse baseret på et generelt modelgrundlag. I praksis vil specialtrænede modeller give bedre performance, især hvis man fokuserer på de objekttyper robotarmen konkret skal sortere eksempelvis specialfremstillede dele, som ikke er letgenkendelige. Dette ses eksempelvis i testen af AI-klassifikation, hvor AI'en fejler ved genkendelse af en 3D printet klods.

Disse observationer viser, at der både er klare styrker og udviklingspotentialer i projektets nuværende form.

8 Konklusion

Projektet har demonstreret, hvordan en robotarm baseret på 3D-print og lavpris-servomotorer kan konstrueres, styres og testes med både manuelle og AI-baserede input. Ved hjælp af inverse kinematik og URDF-modellering er det lykkedes at få armen til at udføre præcise bevægelser, og tests har vist, at systemet opnår en gennemsnitlig gentagelsespræcision på under én millimeter under kontrollerede forhold. Samtidig viser sammenspillet med OpenAI en god forudsætning for identifikation af diverse objekter i forhold til sortering, dog med udfordringer i form af special dele.

Projektet har tydeliggjort en række begrænsninger i både mekanik og software, særligt relateret til gribemekanismens modellering, servomotorernes præcision og den generelle slør i konstruktionen. Disse har direkte påvirket armens evne til at løfte objekter og til at fastholde ensartet præcision i mere komplekse scenarier.

Det er konklusionen, at systemet i sin nuværende form egner sig til demonstration og udvikling af grundlæggende robotfunktioner, men at flere forbedringer, herunder udskiftning af motorer, forbedret kalibrering, samt integration af specialiserede AI-modeller er nødvendige for at kunne anvende systemet i mere krævende eller varierende kontekster.

Projektet har således både opfyldt sit mål om at bygge en fungerende robotarm og har samtidig skabt grobund for en række fremtidige forbedringer og videreudvikling.

References

1. Oludele Mayowa Solaja OKO and Amoo OF. Empowering informal plastic recyclers. Accessed: 19/04-25. 2025. Available from: <https://bmccenvsci.biomedcentral.com/articles/10.1186/s44329-024-00011-5>
2. FSB. Det er blevet dyrere at slippe af med sit affald. Accessed: 07/06-25. 2025. Available from: <https://www.fsb.dk/nyheder/2025/det-er-blevet-dyrere-at-slippe-af-med-sit-affald/#:~:text=Der%20er%20is%C3%A6r%20prisen%20for%20at%20f%C3%A5,afdelinger%2C%20hvor%20det%20g%C3%A5r%20skidt%20med>
3. Nations U. THE 17 GOALS. Accessed: 19/04-25. 2025. Available from: <https://sdgs.un.org/goals>
4. Labs E. Zen Robotics. Accessed: 16/06-25. 2025. Available from: <https://www.everestlabs.ai>
5. Robotics Z. Zen Robotics. Accessed: 16/06-25. 2025. Available from: <https://www.terex.com/zenrobotics>
6. ROBOTICS A. AMP One. Accessed: 16/06-25. 2025. Available from: <https://ampsortation.com/products/amp-one>
7. OpenCV. Detection of ArUco Markersl. Accessed: 19/06-25. 2025. Available from: https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html
8. Brawner S. SolidWorks to URDF. Accessed: 14/06-25. 2021. Available from: https://wiki.ros.org/sw_urdf_exporter
9. Andersen MS. Introduction to musculoskeletal modelling. Accessed: 14/06-25. 2021. Available from: <https://www.sciencedirect.com/topics/engineering/revolute-joint#featured-authors>
10. TullyFoote. urdf. Accessed: 30/05-25. 2023. Available from: https://wiki.ros.org/urdf#A.2BAC8-urdf.2BAC8-Model.URDF_Limitations
11. irisdynamics. Forward and Inverse Kinematics: Explained. Accessed: 25/06-25. 2023. Available from: <https://irisdynamics.com/articles/forward-and-inverse-kinematics?>
12. automaticaddison. Homogeneous Transformation Matrices Using Denavit-Hartenberg. Accessed: 21/06-25. 2020. Available from: <https://automaticaddison.com/homogeneous-transformation-matrices-using-denavit-hartenberg/>
13. Prof. Katie Driggs-Campbell CTwPB. Lecture 14: inverse kinematics II. Accessed: 25/06-25. 2021. Available from: <https://publish.illinois.edu/ece470-intro-robotics/files/2021/11/14-lecture.pdf?>
14. Manceron P. IKPy. Accessed: 03/02-25. DOI: 10.5281/zenodo.6551105. Available from: <https://github.com/Phylliade/ikpy>

9 Appendix A

9.1 Bill of Materials (BOM) – Robotic Arm Assembly

Part Number	Component	Qty.	Material
PRJ001	DS3225 25KG Digital Servo	1	Elektronik
PRJ002	DS3230 30KG Digital Servo	2	Elektronik
PRJ003	DS5160 60KG Digital Servo	2	Elektronik
PRJ004	DS5180 80KG Digital Servo	1	Elektronik
PRJ005	Raspberry Pi 4	1	Elektronik
PRJ006	Raspberry Pi Camera Module 3	1	Elektronik
PRJ007	PC strømforsyning - 230W	1	Elektronik
PRJ008	FSR06BE - 5KG Force Sensor	1	Elektronik
PRJ009	VL53L1X Laser ToF Sensor	1	Elektronik
PRJ010	DC 9A 300W Buck Konverter	3	Elektronik
PRJ011	ESP8266-12E WiFi Module	1	Elektronik
PRJ012	Arduino Uno Rev3	1	Elektronik
PRJ013	Breadboard - Mini	1	Elektronik
PRJ014	Resistor 10K Ω	1	Elektronik
PRJ015	Resistor 1K Ω	1	Elektronik
PRJ016	Resistor 2K Ω	2	Elektronik
PRJ017	Samlemuffe 2 ports	2	Elektronik
PRJ018	Samlemuffe 3 ports	3	Elektronik
PRJ019	Samlemuffe 5 ports	3	Elektronik
PRJ020	Kamera Base	1	PLA
PRJ021	Raspberry Pi / Camera Module Holder	1	PLA
PRJ022	PC strømforsyning Base	1	PLA
PRJ023	Buck Konverter Base	3	PLA
PRJ024	Robot Arm Base	1	PLA
PRJ025	Rotationsflade	1	PLA
PRJ026	Arm 80KG	1	PLA
PRJ027	Arm 60KG	1	PLA
PRJ028	Håndflade	1	PLA
PRJ029	Finger	2	PLA
PRJ030	Tandhjul	2	PLA
PRJ031	Fingerled	2	PLA
PRJ032	Fingerspids	2	PLA
PRJ033	Silicone – Diameter 24	2	Gummi
PRJ034	1x1m Træplade	1	Træ

PRJ035	M8x10 Bolt	22	Metal
PRJ036	M8x25 Bolt	4	Metal
PRJ037	M8x40 Bolt	8	Metal
PRJ038	M4x15 Bolt	8	Metal
PRJ039	M3x10 Bolt	13	Metal
PRJ040	M3x15 Bolt	4	Metal
PRJ041	M3x20 Bolt	4	Metal
PRJ042	M8 Møtrik	28	Metal
PRJ043	M8 Låsemøtrik	6	Metal
PRJ044	M4 Møtrik	8	Metal
PRJ045	M3 Møtrik	8	Metal
PRJ046	M8 Spændeskiver	12	Metal
PRJ047	M4 Spændeskiver	8	Metal
PRJ048	M3 Spændeskiver	4	Metal

9.2 Arbejdstegninger

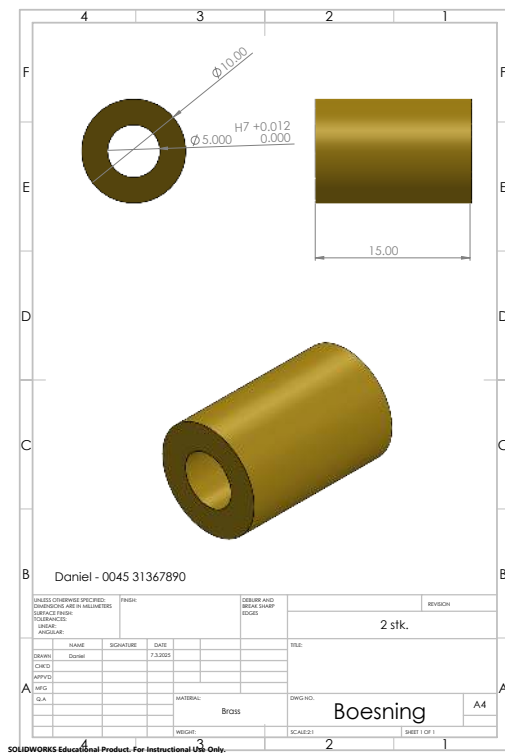


Figure 13: Bøsning

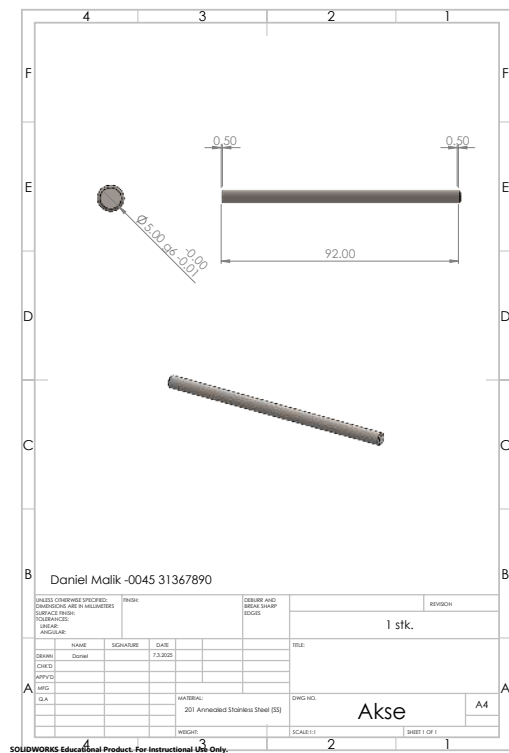


Figure 14: Akse

10 Appendix B

10.1 Forsøgsresultater

10.1.1 Præcision

Måling	a_{venstre} [mm]	$a_{\text{højre}}$ [mm]	δ [mm]	Retning
1	40	60	20	Venstre
2	60	40	-20	Højre
3	65	35	-30	Højre
4	55	45	-10	Højre
5	55	45	-10	Højre
6	50	50	0	Ingen
7	30	70	40	Venstre
8	45	55	10	Venstre
9	50	50	0	Ingen
10	40	60	20	Venstre
11	30	70	40	Venstre
12	30	70	40	Venstre
13	40	60	20	Venstre
14	40	60	20	Venstre
15	50	50	0	Ingen

Table 7: Resultater fra målinger af afstanden fra klodsens centrum til hver gribe-arm, og beregnet forskydning.

10.1.2 Gentagelsestest

Måling	x_i [mm]	y_i [mm]	δ [mm]
1	34.4	28.9	1.94
2	34.2	29.9	1.02
3	33.9	30.1	0.36
4	34.0	30.1	0.10
5	34.1	30.1	0.10
6	34.0	30.2	0.14
7	34.0	30.1	0.10
8	34.2	30.3	0.28
9	34.3	30.2	0.14
10	34.2	30.2	0.10

Table 8: Afstand fra hvert punkt til referencepunktet: Måling 1 til (35.5, 27.3), og herefter hver måling relativt til den foregående.

10.1.3 Klassifikation

Faktiske Objekt	Klassificeret objekt	Faktisk kategori	Klassificeret som	Korrekt (Ja/Nej)
PS4-controller	Controller	E-Waste	E-Waste	Ja
Mus	Mus	E-Waste	E-Waste	Ja
Lommeregner	Lommeragner	E-Waste	E-Waste	Ja
Opladerkabel	Opladerkabel	E-Waste	E-Waste	Ja
Lighter	USB-stik	Ikke E-Waste	E-Waste	Nej
Målebånd	Målebånd	Ikke E-Waste	Ikke E-Waste	Ja
Skruetrækker	Skruetrækker	Ikke E-Waste	Ikke E-Waste	Ja
Svensknøgle	Svensknøgle	Ikke E-Waste	Ikke E-Waste	Ja
Kuglepen	Kuglepen	Ikke E-Waste	Ikke E-Waste	Ja
Tomt iPhone cover	Telefon cover	Ikke E-Waste	Ikke E-Waste	Ja
Vægt	Vægt	E-Waste	E-Waste	Ja
3D-printet objekt	Enhed	Ikke E-Waste	E-Waste	Nej
Arduino board	Elektrisk board	E-Waste	E-Waste	Ja
ESP32	Elektrisk board	E-Waste	E-Waste	Ja
Buckconverter	Elektrisk board	E-Waste	E-Waste	Ja

Table 9: Forsøgsresultater fra klassifikationstest med navngivne objekter.

11 Appendix C

11.1 Prototyper

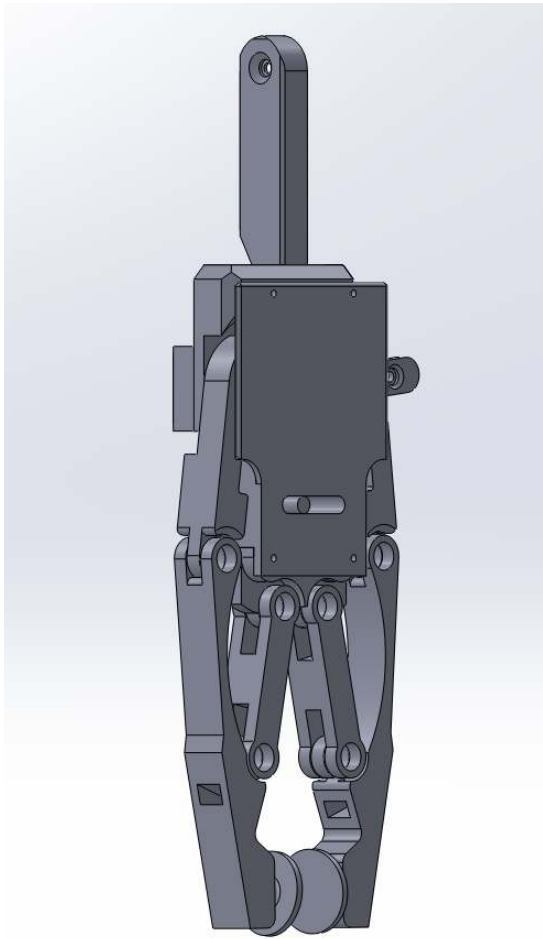


Figure 15: Robothånd



Figure 16: Robothånd exploded view

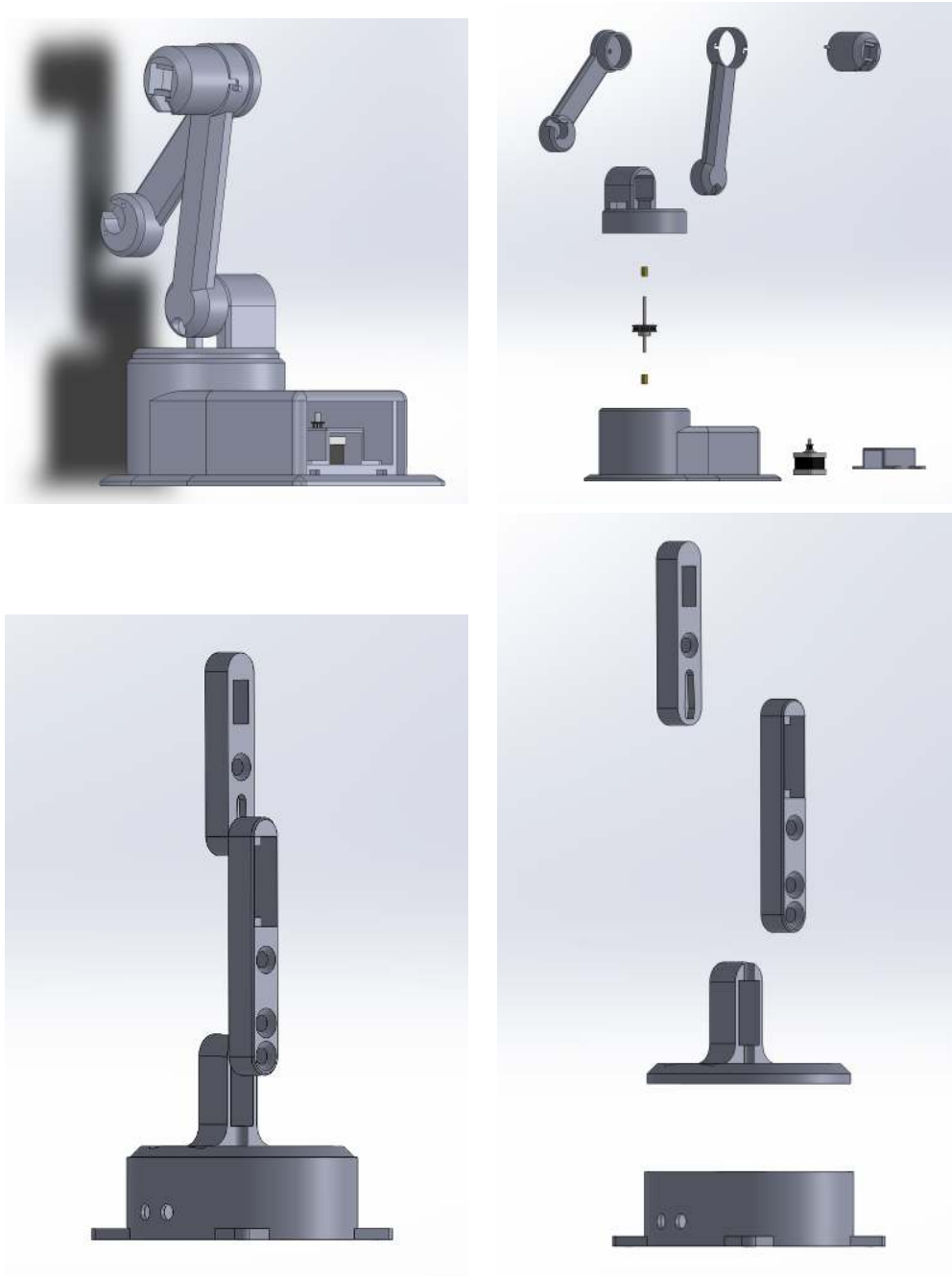


Figure 17: De 2 Prototypes for Arm designet

12 Appendix D

12.1 Koder

Listing 1: C++ code for ESP8266

```
1 #include <Arduino.h>
2 #include <ESP8266WiFi.h>
3 #include <PubSubClient.h>
4 #include <ArduinoJson.h>
```

```
5 #include <Servo.h>
6 #include <SoftwareSerial.h>
7
8 // SoftwareSerial RX and TX pins
9 #define RXD2 13
10 #define TXD2 12
11
12 const char *ssid = "N - iPhone";
13 const char *password = "nick12345";
14 const char *mqttServer = "172.20.10.3";
15 const int mqttPort = 1883;
16
17 const char *mqttTopics[] = {
18     "esp32/receive-AI",
19     "esp32/receive-IK"};
20 const int topicCount = sizeof(mqttTopics) / sizeof(mqttTopics[0]);
21
22 const int motorPins[4] = {4, 14, 15, 16}; // Avoid GPIO12, 13
23 Servo motors[4];
24
25 WiFiClient espClient;
26 PubSubClient client(espClient);
27
28 // Serial connection to Arduino
29 SoftwareSerial mySerial(RXD2, TXD2); // RX, TX
30
31 void callback(char *topic, byte *message, unsigned int length)
32 {
33     Serial.printf("\n Message received on topic: %s\n", topic);
34
35     if (length == 0 || length >= 256)
36     {
37         Serial.println("Invalid or oversized JSON");
38         return;
39     }
40
41     char jsonBuffer[256];
42     memcpy(jsonBuffer, message, length);
43     jsonBuffer[length] = '\0';
44
45     Serial.print("Payload: ");
46     Serial.println(jsonBuffer);
```

```

47
48 StaticJsonDocument<256> doc;
49 DeserializationError error = deserializeJson(doc, jsonBuffer);
50 if (error)
51 {
52     Serial.print("JSON parse error: ");
53     Serial.println(error.c_str());
54     return;
55 }
56
57 if (strcmp(topic, "esp32/receive-AI") == 0)
58 {
59     const char *objectName = doc["objectName"] | "Unknown";
60     float objectHeight = doc["objectHeight"] | -1.0;
61     float gripForce = doc["gripForce"] | -1.0;
62     int analogForceTargetVal = doc["analogForceTargetVal"] | -1;
63
64     int m1 = doc["motor1"] | -1;
65     int m2 = doc["motor2"] | -1;
66     int m3 = doc["motor3"] | -1;
67     int m4 = doc["motor4"] | -1;
68
69     const char *isEWaste = doc["isEWaste"] | "unknown"; // New field
70
71     Serial.println("AI Estimations Received:");
72     Serial.printf("Object Name: %s\n", objectName);
73     Serial.printf("Object Height: %.2f cm\n", objectHeight);
74     Serial.printf("Target Grip Force: %.2f N\n", gripForce);
75     Serial.printf("Analog Target Value: %d\n", analogForceTargetVal);
76     Serial.printf("Motor Angles: %d, %d, %d, %d\n", m1, m2, m3, m4);
77     Serial.printf("Is Electronic Waste: %s\n", isEWaste); // New log
78
79     String summary = String(m1) + "," +
80                     String(m2) + "," +
81                     String(m3) + "," +
82                     String(m4) + "," +
83                     String(analogForceTargetVal) + "," +
84                     String(isEWaste);
85
86     const char *command_to_arduino = "DoAI";
87     mySerial.println("DoAI," + summary);
88

```

```

89 // mySerial.println(command_to_arduino);
90 // delay(1000); // Give time before next line
91 // mySerial.println(summary);
92
93 Serial.print("Sent to Arduino: ");
94 Serial.println(summary);
95 }
96 else if (strcmp(topic, "esp32/receive-IK") == 0)
97 {
98     String toArduino = "";
99
100     for (int i = 0; i < 4; i++)
101     {
102         String key = "motor" + String(i + 1);
103         if (doc.containsKey(key))
104         {
105             float angle = doc[key].as<float>();
106             motors[i].write(angle);
107             toArduino += String(angle, 2);
108             Serial.printf("%s = %.2f\n", key.c_str(), angle);
109         }
110         else
111         {
112             toArduino += "0.00";
113             Serial.printf("Missing %s\n", key.c_str());
114         }
115
116         if (i < 3)
117             toArduino += ",";
118     }
119
120     const char *isEWaste = doc["isEWaste"] | "unknown";
121     Serial.printf("Is Electronic Waste: %s\n", isEWaste);
122
123     mySerial.println("DoHuman," + toArduino + "," + String(isEWaste)); // Send
        all
124     Serial.print("Sent to Arduino: ");
125     Serial.println(toArduino + "," + isEWaste);
126 }
127 }
128
129 void connectToWiFi()

```

```
130 {
131   WiFi.mode(WIFI_STA);
132   WiFi.begin(ssid, password);
133   Serial.print("Connecting to WiFi");
134
135   for (int i = 0; i < 30 && WiFi.status() != WL_CONNECTED; i++)
136   {
137     delay(500);
138     Serial.print(".");
139   }
140
141   if (WiFi.status() == WL_CONNECTED)
142   {
143     Serial.println("\n WiFi connected");
144     Serial.print("IP: ");
145     Serial.println(WiFi.localIP());
146   }
147   else
148   {
149     Serial.println("\nWiFi failed. Restarting...");
150     delay(1000);
151     ESP.restart();
152   }
153 }
154
155 void connectToMQTT()
156 {
157   delay(500); // Ensure WiFi is fully stable
158
159   Serial.println("Connecting to MQTT...");
160   for (int i = 0; i < 5; i++)
161   {
162     if (client.connect("ESP8266Client"))
163     {
164       Serial.println("MQTT connected");
165       for (int j = 0; j < topicCount; j++)
166       {
167         client.subscribe(mqttTopics[j]);
168         Serial.printf("Subscribed: %s\n", mqttTopics[j]);
169       }
170       return;
171     }
```

```
172     else
173     {
174         Serial.printf("MQTT failed (rc=%d). Retry...\n", client.state());
175         delay(1000);
176     }
177 }
178
179 Serial.println("MQTT retries failed. Will retry in loop()");
180 }
181
182 void setup()
183 {
184     delay(500);
185     Serial.begin(115200);
186     mySerial.begin(9600);
187     delay(1000);
188     Serial.println("ESP8266 booting...");
189
190     for (int i = 0; i < 4; i++)
191     {
192         motors[i].attach(motorPins[i], 500, 2400); // Default 50Hz
193     }
194
195     client.setServer(mqttServer, mqttPort);
196     client.setCallback(callback);
197
198     connectToWiFi();
199     connectToMQTT();
200 }
201
202 void loop()
203 {
204     if (!client.connected())
205     {
206         connectToMQTT();
207     }
208     client.loop();
209
210     // Listen for Arduino responses
211     if (mySerial.available())
212     {
213         String arduinoMessage = mySerial.readStringUntil('\n');
```

```
214     arduinoMessage.trim();
215
216     if (arduinoMessage == "AiFailed")
217     {
218         Serial.println("Arduino reported: AI failed.");
219
220         client.publish("esp32/ai-status", "AiFailed"); // Optional: publish to
                MQTT
221     }
222     else
223     {
224         Serial.print("From Arduino: ");
225         Serial.println(arduinoMessage);
226     }
227 }
228 }
```

Listing 2: Code for Arduino Uno

```
1 #include <SoftwareSerial.h>
2 #include <Servo.h>
3
4 Servo servo1, servo2, servo3, servo4, servo5;
5 int current1 = 66;
6 int current2 = 165;
7 int current3 = 10;
8 int current4 = 141;
9 int current5 = 90;
10 int stepDelay = 20;
11
12 const int fsrPin = A0;
13
14 SoftwareSerial espSerial(5, 6); // RX, TX
15
16 bool awaitingHumanValues = false;
17 int humanAngles[4];
18 String sortingCategory = "";
19
20 bool awaitingAiValues = false;
21 int aiAngles[4];
22 int aiTargetForce = -1;
23
24 String humanIsEWaste = "";
```

```
25 String aiIsEWaste = ""; // stores "yes" or "no"
26
27 void setup() {
28     Serial.begin(9600);
29     espSerial.begin(9600);
30
31     servo1.attach(13);
32     servo2.attach(11);
33     servo3.attach(8);
34     servo4.attach(12);
35     servo5.attach(10);
36
37     servo1.write(current1);
38     servo2.write(current2);
39     servo3.write(current3);
40     servo4.write(current4);
41     servo5.write(current5);
42
43     Serial.println("Arduino Ready");
44 }
45 void loop() {
46     while (espSerial.available()) {
47         String input = espSerial.readStringUntil('\n');
48         input.trim();
49
50         Serial.print("Received: ");
51         Serial.println(input);
52
53         input.replace("\r", ""); // Remove carriage returns if any
54
55         String values[7]; // Up to 7 parts
56         int index = 0;
57
58         while (input.length() > 0 && index < 7) {
59             int comma = input.indexOf(',');
60             String val = (comma == -1) ? input : input.substring(0, comma);
61             input = (comma == -1) ? "" : input.substring(comma + 1);
62             val.trim();
63             values[index++] = val;
64         }
65
66         if (index < 2) {
```

```
67     Serial.println("Not enough data.");
68     return;
69 }
70
71 String command = values[0];
72
73 if (command == "DoAI" && index == 7) {
74     aiAngles[0] = values[1].toInt();
75     aiAngles[1] = values[2].toInt();
76     aiAngles[2] = values[3].toInt();
77     aiAngles[3] = values[4].toInt();
78     aiTargetForce = values[5].toInt();
79     aiIsEWaste = values[6];
80
81     Serial.println("AI values parsed:");
82     Serial.print("Angles: ");
83     Serial.print(aiAngles[0]); Serial.print(", ");
84     Serial.print(aiAngles[1]); Serial.print(", ");
85     Serial.print(aiAngles[2]); Serial.print(", ");
86     Serial.println(aiAngles[3]);
87     Serial.print("Target Force: "); Serial.println(aiTargetForce);
88     Serial.print("isEWaste: "); Serial.println(aiIsEWaste);
89
90     executeAiRoutine();
91 }
92 else if (command == "DoHuman" && index == 6) {
93     humanAngles[0] = values[1].toInt();
94     humanAngles[1] = values[2].toInt();
95     humanAngles[2] = values[3].toInt();
96     humanAngles[3] = values[4].toInt();
97     humanIsEWaste = values[5];
98
99     Serial.println("Human values parsed:");
100    Serial.print("Angles: ");
101    Serial.print(humanAngles[0]); Serial.print(", ");
102    Serial.print(humanAngles[1]); Serial.print(", ");
103    Serial.print(humanAngles[2]); Serial.print(", ");
104    Serial.println(humanAngles[3]);
105    Serial.print("isEWaste: "); Serial.println(humanIsEWaste);
106
107    executeHumanRoutine();
108 }
```

```
109     else {
110         Serial.println("Unrecognized or malformed command.");
111     }
112 }
113 }
114
115
116 void executeAiRoutine() {
117
118     // Step 1: Move to AI angles
119     moveServoSmooth(servo1, current1, aiAngles[0]); current1 = aiAngles[0];
120     moveServoSmooth(servo2, current2, aiAngles[1]); current2 = aiAngles[1];
121     moveServoSmooth(servo3, current3, aiAngles[2]); current3 = aiAngles[2];
122     moveServoSmooth(servo4, current4, aiAngles[3]); current4 = aiAngles[3];
123
124     // Step 2: Close gripper until force is reached
125     bool gripSuccess = false;
126     for (int angle = current5; angle >= 0; angle--) {
127         servo5.write(angle);
128         delay(stepDelay);
129
130         int force = analogRead(fsrPin);
131         Serial.print("Force: ");
132         Serial.println(force);
133
134         if (force >= aiTargetForce) {
135             current5 = angle;
136             gripSuccess = true;
137             break;
138         }
139     }
140
141     // Step 2b: Gripper failed -- force not reached
142     if (!gripSuccess) {
143         Serial.println("Grip failed -- force not reached");
144
145         moveServoSmooth(servo5, current5, 90); current5 = 90;
146         moveServoSmooth(servo2, current2, 165); current2 = 165;
147         moveServoSmooth(servo1, current1, 66); current1 = 66;
148         moveServoSmooth(servo3, current3, 10); current3 = 10;
149         moveServoSmooth(servo4, current4, 141); current4 = 141;
150     }
```

```

151     espSerial.println("AiFailed");
152     Serial.println("Sent 'AiFailed' to ESP8266");
153     return;
154 }
155
156 // Step 3: Wait 1 sec
157 delay(1000);
158
159 // Step 4: Return to rest position
160 moveServoSmooth(servo2, current2, 165); current2 = 165;
161 moveServoSmooth(servo1, current1, 66); current1 = 66;
162 moveServoSmooth(servo3, current3, 10); current3 = 10;
163 moveServoSmooth(servo4, current4, 78); current4 = 78;
164
165 // Step 5: Wait 1 sec
166 delay(2000);
167
168 // Step 6: Check force again
169 int finalForce = analogRead(fsrPin);
170 Serial.print("Final Force Check: ");
171 Serial.println(finalForce);
172
173 if (finalForce >= aiTargetForce) {
174     // Step 7a: Go back to AI angles
175
176     box location
177
178     if (aiIsEWaste == "yes") {
179         moveServoSmooth(servo1, current1, 22); current1 = 22;
180         moveServoSmooth(servo3, current3, 68); current3 = 68;
181         moveServoSmooth(servo4, current4, 141); current4 = 141;
182         moveServoSmooth(servo2, current2, 112); current2 = 112;
183         moveServoSmooth(servo4, current4, 102); current4 = 102;
184     } else {
185         moveServoSmooth(servo1, current1, 36); current1 = 36;
186         moveServoSmooth(servo3, current3, 36); current3 = 36;
187         moveServoSmooth(servo4, current4, 141); current4 = 141;
188         moveServoSmooth(servo2, current2, 84); current2 = 84;
189         moveServoSmooth(servo4, current4, 129); current4 = 129;
190     }

```

Change this to

```

191 // Step 8a: Open gripper
192 moveServoSmooth(servo5, current5, 90); current5 = 90;
193
194 // Step 9a: Return to rest
195 moveServoSmooth(servo2, current2, 165); current2 = 165;
196 moveServoSmooth(servo1, current1, 66); current1 = 66;
197 moveServoSmooth(servo3, current3, 10); current3 = 10;
198 moveServoSmooth(servo4, current4, 141); current4 = 141;
199
200 Serial.println("AI sequence complete.");
201 } else {
202     moveServoSmooth(servo3, current3, 10); current3 = 10;
203     espSerial.println("AiFailed");
204     Serial.println("AI force insufficient. Sent AiFailed.");
205     moveServoSmooth(servo4, current4, 141); current4 = 141;
206 }
207 }
208
209
210 void executeHumanRoutine() {
211     // Step 1: Move to received human angles
212     moveServoSmooth(servo5, current5, 90); current5 = 90;
213     moveServoSmooth(servo1, current1, humanAngles[0]); current1 = humanAngles[0];
214     moveServoSmooth(servo2, current2, humanAngles[1]); current2 = humanAngles[1];
215     moveServoSmooth(servo3, current3, humanAngles[2]); current3 = humanAngles[2];
216     moveServoSmooth(servo4, current4, humanAngles[3]); current4 = humanAngles[3];
217
218     // Step 2: Close gripper until force >= 800 OR angle reaches 40
219     bool gripSuccess = false;
220     for (int angle = current5; angle >= 0; angle--) {
221         servo5.write(angle);
222         delay(stepDelay);
223
224         int force = analogRead(fsrPin);
225         Serial.print("Human Grip Force: ");
226         Serial.println(force);
227
228         if (force >= 100) {
229             current5 = angle;
230             gripSuccess = true;
231             break;
232         }

```

```

233 }
234
235 // If force was NOT reached, treat as failure and abort
236 if (!gripSuccess) {
237     Serial.println("Force not reached -- human grip aborted.");
238
239     moveServoSmooth(servo5, current5, 90); current5 = 90;
240     moveServoSmooth(servo2, current2, 165); current2 = 165;
241     moveServoSmooth(servo1, current1, 66); current1 = 66;
242     moveServoSmooth(servo3, current3, 10); current3 = 10;
243     moveServoSmooth(servo4, current4, 141); current4 = 141;
244
245     return;
246 }
247
248 // Step 3: Move to 66, 165, 10, 78 -- gripper stays
249 moveServoSmooth(servo1, current1, 66); current1 = 66;
250 moveServoSmooth(servo2, current2, 165); current2 = 165;
251 moveServoSmooth(servo3, current3, 10); current3 = 10;
252 moveServoSmooth(servo4, current4, 78); current4 = 78;
253
254 // Step 4: Wait and read force once
255 delay(1000);
256 int finalForce = analogRead(fsrPin);
257 Serial.print("Final Human Force Check: ");
258 Serial.println(finalForce);
259
260 if (finalForce > 0) {
261     // Step 5a: Go back to received angles, then open gripper
262     CHANGE THIS TO BOX ANGLES
263     if (humanIsEWaste == "yes") {
264         moveServoSmooth(servo1, current1, 22); current1 = 22;
265         moveServoSmooth(servo3, current3, 68); current3 = 68;
266         moveServoSmooth(servo4, current4, 141); current4 = 141;
267         moveServoSmooth(servo2, current2, 112); current2 = 112;
268         moveServoSmooth(servo4, current4, 102); current4 = 102;
269     } else {
270         moveServoSmooth(servo1, current1, 36); current1 = 36;
271         moveServoSmooth(servo3, current3, 36); current3 = 36;
272         moveServoSmooth(servo4, current4, 141); current4 = 141;
273         moveServoSmooth(servo2, current2, 84); current2 = 84;
274         moveServoSmooth(servo4, current4, 129); current4 = 129;

```

```
274     }
275
276     moveServoSmooth(servo5, current5, 90); current5 = 90;
277
278     Serial.println("Human hand detected, object released.");
279
280     moveServoSmooth(servo2, current2, 165); current2 = 165;
281     moveServoSmooth(servo1, current1, 66); current1 = 66;
282     moveServoSmooth(servo3, current3, 10); current3 = 10;
283     moveServoSmooth(servo4, current4, 141); current4 = 141;
284
285
286
287 } else {
288     // Step 5b: Go back to rest pose
289     moveServoSmooth(servo2, current2, 165); current2 = 165;
290     moveServoSmooth(servo1, current1, 66); current1 = 66;
291     moveServoSmooth(servo3, current3, 10); current3 = 10;
292     moveServoSmooth(servo4, current4, 141); current4 = 141;
293
294     moveServoSmooth(servo5, current5, 90); current5 = 90;
295
296     Serial.println("No hand detected -- returned to rest.");
297 }
298 }
299
300
301
302
303 void moveServoSmooth(Servo &servo, int fromAngle, int toAngle) {
304     if (fromAngle < toAngle) {
305         for (int angle = fromAngle; angle <= toAngle; angle++) {
306             servo.write(angle);
307             delay(stepDelay);
308         }
309     } else {
310         for (int angle = fromAngle; angle >= toAngle; angle--) {
311             servo.write(angle);
312             delay(stepDelay);
313         }
314     }
315 }
```

Listing 3: Main file for Raspberry Pi

```

1 import threading
2 import paho.mqtt.client as mqtt
3 import time
4 import cv2
5 import numpy as np
6 import subprocess
7
8
9 should_run = True
10
11 # --- MQTT Listener Setup ---
12 def on_message(client, userdata, msg):
13     global should_run
14
15     payload = msg.payload.decode()
16     print(f"MQTT message on topic: {msg.topic} | payload: {payload}")
17     print("on_message() was triggered")
18
19     if msg.topic == "esp32/ai-status" and payload.strip() == "AiFailed":
20         print("AI failed -- re-running detection pipeline")
21
22         try:
23             x, y = detect_and_measure(USE_CAMERA=True)
24
25             if x is None or y is None:
26                 raise ValueError("detect_and_measure() returned None for x or y")
27
28             target = [abs(y), -x, 0.01]
29             angle1, angle2, angle3, angle4 = do_IK.compute_scaled_angles(target)
30
31             objectName, objectHeight, gripForce, isEWaste = Ai_stuff.
                analyze_image("/home/nicksrasp/Desktop/largest_object.jpg")
32
33             import send_data
34             send_data.send_motor_angles(angle1, angle2, angle3, angle4,
                isEWaste)
35
36             should_run = False
37

```

```
38     except Exception as e:
39         print("Error during AI re-run:", e)
40
41
42
43
44 def start_mqtt_listener():
45     client = mqtt.Client()
46     MQTT_BROKER_IP = "172.20.10.3" # same as ESP8266 mqttServer
47     client.connect(MQTT_BROKER_IP, 1883, 60)
48     client.subscribe("esp32/ai-status")
49     client.on_message = on_message
50     print("Subscribed to 'esp32/ai-status'")
51     client.loop_forever()
52
53 # Start MQTT listener in a background thread
54 threading.Thread(target=start_mqtt_listener, daemon=True).start()
55
56 def detect_and_measure(USE_CAMERA=True): # WORKS
57
58     print("now inside detect and measure")
59
60     image_path = "captured_image.jpg"
61     existing_image_path = "/home/nicksrasp/Desktop/captured_image.jpg"
62
63     mm_per_px_x = 0.416
64     mm_per_px_y = 0.412
65     offset_up_mm = 733.29
66     offset_left_mm = 490
67     line_offset_mm = 610
68
69     if USE_CAMERA:
70         print("Capturing image...")
71         subprocess.run([
72             "libcamera-still", "-n", "-o", image_path,
73             "--width", "4608", "--height", "2592", "--quality", "100", "-t", "
74             1000"
75         ], check=True)
76     else:
77         print("Using existing image...")
78         image_path = existing_image_path
```

```

79     image = cv2.imread(image_path)
80     if image is None:
81         raise ValueError("Could not load image: " + image_path)
82     gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
83
84     aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
85     parameters = cv2.aruco.DetectorParameters()
86     detector = cv2.aruco.ArucoDetector(aruco_dict, parameters)
87     corners, ids, _ = detector.detectMarkers(gray)
88
89     if ids is None or not (0 in ids and 2 in ids):
90         raise ValueError("ArUco markers 0 and 2 not found.")
91
92     def get_corner(marker_id, idx):
93         i = np.where(ids == marker_id)[0][0]
94         return tuple(map(int, corners[i][0][idx]))
95
96     x1, y1 = get_corner(0, 0)
97     x2, y2 = get_corner(2, 2)
98     x_min, x_max = sorted([x1, x2])
99     y_min, y_max = sorted([y1, y2])
100     cropped = image[y_min:y_max, x_min:x_max].copy()
101
102     br_x, br_y = get_corner(0, 2)
103     br_x -= x_min
104     br_y -= y_min
105     dot_x = int(br_x - offset_left_mm / mm_per_px_x)
106     dot_y = int(br_y - offset_up_mm / mm_per_px_y)
107     line_y = int(br_y - line_offset_mm / mm_per_px_y)
108
109     # Crop everything left of the vertical line
110     crop_x = max(int(dot_x - 30 / mm_per_px_x), 0)
111     cropped = cropped[:, crop_x:]
112     dot_x -= crop_x
113     br_x -= crop_x # Adjust other references too
114
115     clean_cropped = cropped.copy() # Save clean copy BEFORE any drawings
116
117     # Draw overlays
118     cv2.circle(cropped, (dot_x, dot_y), 10, (0, 0, 255), -1)
119     cv2.line(cropped, (0, line_y), (cropped.shape[1], line_y), (0, 255, 255),
120             2)

```

```

120
121 gray_crop = cv2.cvtColor(clean_cropped, cv2.COLOR_BGR2GRAY)
122 _, binary = cv2.threshold(gray_crop, 60, 255, cv2.THRESH_BINARY_INV)
123 contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.
    CHAIN_APPROX_SIMPLE)
124
125 debug_img = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)
126 cv2.drawContours(debug_img, contours, -1, (0, 255, 0), 1)
127 cv2.line(debug_img, (0, line_y), (debug_img.shape[1], line_y), (0, 255,
    255), 2)
128 cv2.imwrite("cropped_contours_debug.jpg", debug_img)
129
130 largest, max_area, center = None, 0, None
131 for c in contours:
132     M = cv2.moments(c)
133     if M["m00"] == 0:
134         continue
135     cx, cy = int(M["m10"] / M["m00"]), int(M["m01"] / M["m00"])
136     if cy <= line_y:
137         continue
138     area = cv2.contourArea(c)
139     if area > max_area:
140         largest, max_area, center = c, area, (cx, cy)
141
142 if largest is not None:
143     box = cv2.boxPoints(cv2.minAreaRect(largest)).astype(int)
144     cv2.drawContours(cropped, [box], 0, (0, 255, 0), 2)
145     cv2.circle(cropped, center, 5, (255, 0, 0), -1)
146     cv2.arrowedLine(cropped, (dot_x, dot_y), center, (255, 0, 255), 2)
147
148     dx_m = ((center[0] - dot_x) * mm_per_px_x) / 1000
149     dy_m = -((center[1] - dot_y) * mm_per_px_y) / 1000
150     print(f"dx = {dx_m:.3f} m, dy = {dy_m:.3f} m")
151     text = f"x: {dx_m:.3f} m, y: {dy_m:.3f} m"
152     cv2.putText(cropped, text, (center[0] + 10, center[1] - 10), cv2.
        FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 0), 2)
153
154     # --- Clean crop around the object with margin ---
155     x, y, w, h = cv2.boundingRect(largest)
156     margin = 100 # pixels
157     x1_crop = max(x - margin, 0)
158     y1_crop = max(y - margin, 0)

```

```

159     x2_crop = min(x + w + margin, clean_cropped.shape[1])
160     y2_crop = min(y + h + margin, clean_cropped.shape[0])
161
162     object_crop = clean_cropped[y1_crop:y2_crop, x1_crop:x2_crop]
163     cv2.imwrite("largest_object.jpg", object_crop)
164     print("Saved largest object crop: largest_object.jpg")
165 else:
166     print("No object found below the yellow line.")
167     dx_m, dy_m = None, None
168
169     cv2.imwrite("final_output.jpg", cropped)
170     print("Saved:")
171     print(f" - {image_path}")
172     print(" - final_output.jpg")
173     print(" - cropped_contours_debug.jpg")
174
175     return round(dx_m, 2), round(dy_m, 2)
176
177
178
179
180
181 # Run it like this:
182 x,y = detect_and_measure(USE_CAMERA=True) # for new picture
183 #x, y = detect_and_measure(USE_CAMERA=False) # for existing image
184
185 print(x, y)
186
187 # ----- > OpenAI
188
189 import Ai_stuff
190
191 objectName, objectHeight, gripForce, isEWaste = Ai_stuff.analyze_image("/home/
    nicksrasp/Desktop/largest_object.jpg")
192 print("Main Object:", objectName)
193 print("Estimated Height:", float(objectHeight), "m")
194 print("Estimated needed force:", float(gripForce), "N")
195 print("Is it electronics waste:", isEWaste)
196
197 # ----- > INVERSE
198
    KINEMATICS

```

```

199 import do_IK
200
201
202 target = [abs(y), -x, float(objectHeight)/2]
203 # target = [abs(y), -x, 0.03]
204 # target = [abs(y), -x, 0.1]
205
206
207 angle1, angle2, angle3, angle4 = do_IK.compute_scaled_angles(target)
208
209
210
211 # ----- > SEND TO
    ARDUINO
212
213 import send_data
214 def force_to_analog(force):
215     if force <= 0:
216         return 0
217     analog = 800 * (force / 10.0) ** (1 / 2.5)
218     # return min(round(analog), 800)
219     return round(analog)
220
221
222
223 analog_force_target_val = force_to_analog(float(gripForce)/2)
224 print("This is analog force value", analog_force_target_val)
225
226 send_data.send_AI_estimations(objectName, objectHeight, gripForce,
    analog_force_target_val, angle1, angle2, angle3, angle4, isEWaste)
227
228
229
230
231 while should_run:
232     time.sleep(1)
233
234 print("Done. Exiting program.")

```

Listing 4: Send Data file for raspberry pi

```

1 import paho.mqtt.client as mqtt
2 import json

```

```
3
4 def send_motor_angles(m1, m2, m3, m4, isEWaste, broker="localhost", topic="
    esp32/receive-IK", port=1883):
5     """
6     Sends motor angles via MQTT.
7
8     Parameters:
9         m1 to m5 (int or str): Angles for motors 1 to 4.
10        broker (str): MQTT broker address.
11        topic (str): MQTT topic to publish to.
12        port (int): MQTT port (default 1883).
13    """
14    angles = {
15        "motor1": m1,
16        "motor2": m2,
17        "motor3": m3,
18        "motor4": m4,
19        "isEWaste": isEWaste
20    }
21
22    message = json.dumps(angles)
23
24    client = mqtt.Client()
25    client.connect(broker, port, 60)
26    client.publish(topic, message)
27    client.disconnect()
28
29    print(f"Sent angles: {message}")
30
31 def send_AI_estimations(objectName, objectHeight, gripForce,
    analogForceTargetVal, m1, m2, m3, m4, isEWaste, broker="localhost", topic="esp32
    /receive-AI", port=1883):
32
33    AI_estimations = {
34        "objectName": objectName,
35        "objectHeight": objectHeight,
36        "gripForce": gripForce,
37        "analogForceTargetVal": analogForceTargetVal,
38        "motor1": m1,
39        "motor2": m2,
40        "motor3": m3,
41        "motor4": m4,
```

```
42     "isEWaste": isEWaste
43
44 }
45
46 message = json.dumps(AI_estimations)
47
48 client = mqtt.Client()
49 client.connect(broker, port, 60)
50 client.publish(topic, message)
51 client.disconnect()
52
53 print(f"Sent AI estimations: {message}")
```

Listing 5: Code file for openAI API

```
1 import os
2 import base64
3 import re
4 from dotenv import load_dotenv
5 import openai
6
7 def analyze_image(image_path: str):
8     load_dotenv()
9     api_key = os.getenv("OPENAI_API_KEY")
10    client = openai.OpenAI(api_key=api_key)
11
12    with open(image_path, "rb") as image_file:
13        base64_image = base64.b64encode(image_file.read()).decode("utf-8")
14
15    response = client.chat.completions.create(
16        model="gpt-4o",
17        messages=[
18            {
19                "role": "user",
20                "content": [
21                    {
22                        "type": "text",
23                        "text": (
24                            "You are an image analysis system trained to make
25                            visual estimations from pictures, even when
26                            uncertain.\n"
27                            "Always provide your best guess based only on the
28                            image. Do NOT say you can't tell.\n"

```

```

26         "Return exactly:\n"
27         "1. Object_type (one word)\n"
28         "2. Object_height (in meters) -- number only, no
           units\n"
29         "3. Estimated grip force (N) to lift the object
           with rubber fingertips and 2.5 cm_2 contact area
           number only, no units\n"
30         "4. Is this object electronic waste? Answer with
           one word only: 'yes' or 'no'. \n"
31         "Make a guess even if you're not sure. Never reply
           with an explanation or apology."
32     ),
33     },
34     {
35         "type": "image_url",
36         "image_url": {"url": f"data:image/jpeg;base64,{
           base64_image}"},
37     },
38     ],
39     }
40 ],
41     max_tokens=300
42 )
43
44 result = response.choices[0].message.content
45 print("RAW RESPONSE:\n", result)
46
47 # Updated regex to match new format
48 object_match = re.search(r"1\.\s*Object_type:\s*(\w+)", result)
49 height_match = re.search(r"2\.\s*Object_height:\s*([\d.]+)", result)
50 force_match = re.search(r"3\.\s*Estimated grip force:\s*([\d.]+)", result)
51 ewaste_match = re.search(r"4\.\s*Is this object electronic waste\?:?\s*(
           yes|no)", result, re.IGNORECASE)
52
53 object_type = object_match.group(1) if object_match else None
54 height_m = float(height_match.group(1)) if height_match else None
55 grip_force = float(force_match.group(1)) if force_match else None
56 is_electronic_waste = ewaste_match.group(1).lower() if ewaste_match else
           None
57
58 return object_type, height_m, grip_force, is_electronic_waste

```

Listing 6: Code for performing inverse kinematics for raspberry pi

```
1
2 import ikpy.chain
3 import math
4
5 def compute_scaled_angles(
6     target_position,
7     urdf_path="/home/nicksrasp/Desktop/tol_arm_2_assem_urdf_4.urdf",
8
9 ):
10     # Load robot chain
11     my_chain = ikpy.chain.Chain.from_urdf_file(
12         urdf_path,
13         active_links_mask=[False, True, True, True, True, True]
14     )
15
16     target_orientation = [0, 0, 0]
17     initial_guesses = [0.0, 0.5, 1.0, 1.5]
18     solution_found = False
19
20     for guess in initial_guesses:
21         initial_position = [0, 0, 0, 0, 0, 0]
22         initial_position[2] = guess
23
24         try:
25             ik = my_chain.inverse_kinematics(
26                 target_position,
27                 target_orientation,
28                 orientation_mode="Y",
29                 initial_position=initial_position
30             )
31
32             upper_arm = ik[2]
33             joint_2 = ik[3]
34             joint_3 = ik[4]
35
36             if (0 <= upper_arm <= 1.57 and
37                 0 <= joint_2 <= 1.57 and
38                 0 <= joint_3 <= 1.57):
39
40                 solution_found = True
41                 break
```

```
42     except Exception as e:
43         print("IKPy error:", e)
44
45     if not solution_found:
46         print("No valid solution found.")
47         return None
48
49     # Convert to degrees
50     angles_deg = list(map(lambda r: math.degrees(r), ik.tolist()))
51     angles = angles_deg[1:5] # joints 1 to 4
52     angles[0] = round(angles[0] * (66/90))
53
54     angles[3] = 159 - abs(round(angles[3] * 0.7278 + 14)) # Joint 4
55     angles[1] = 140 - abs(round(angles[1] * 1.0556 - 25)) # Joint 2
56     angles[2] = round(angles[2] * 1.0222 + 10) # Joint 3
57     print(angles[0], angles[1], angles[2], angles[3])
58
59
60     return angles[0], angles[1], angles[2], angles[3]
```

13 Appendix E

13.1 Styrke Udregninger - Maple

$$g := 9.82 \frac{\text{m}}{\text{s}^2}$$

$$g := 9.82 \frac{\text{m}}{\text{s}^2} \quad (1)$$

Med motorer i rumen 100 g pr motor

$$M = 0.115 \text{ m} \cdot (86 \text{ g} \cdot g) + 0.325 \text{ m} \cdot (77 \text{ g} \cdot g) + 0.23 \text{ m} \cdot (100 \text{ g} \cdot g) + 0.42 \text{ m} \cdot (100 \text{ g} \cdot g) + 0.42 \text{ m} \cdot (300 \text{ g} \cdot g) + 0.42 \text{ m} \cdot (700 \text{ g} \cdot g)$$

$$M = 5.105565300 \text{ J} \quad (2)$$

$$M = 0.095 \text{ m} \cdot (0.77 \text{ g} \cdot g) + 0.19 \text{ m} \cdot (300 \text{ g} \cdot g) + 0.19 \text{ m} \cdot (700 \text{ g} \cdot g)$$

$$M = 1.866518333 \text{ J} \quad (3)$$

77g direkte

$$M = 0.115 \text{ m} \cdot (86 \text{ g} \cdot g) + 0.325 \text{ m} \cdot (77 \text{ g} \cdot g) + 0.23 \text{ m} \cdot (100 \text{ g} \cdot g) + 0.42 \text{ m} \cdot (100 \text{ g} \cdot g) + 0.52 \text{ m} \cdot (300 \text{ g} \cdot g) + 0.62 \text{ m} \cdot (700 \text{ g} \cdot g)$$

$$M = 6.774965300 \text{ J} \quad (4)$$

$$M = 0.095 \text{ m} \cdot (0.77 \text{ g} \cdot g) + 0.19 \text{ m} \cdot (100 \text{ g} \cdot g) + 0.29 \text{ m} \cdot (300 \text{ g} \cdot g) + 0.39 \text{ m} \cdot (700 \text{ g} \cdot g)$$

$$M = 3.722498333 \text{ J} \quad (5)$$

$$M = 0.1 \text{ m} \cdot (300 \text{ g} \cdot g) + 0.2 \text{ m} \cdot (700 \text{ g} \cdot g)$$

$$M = 1.669400000 \text{ J} \quad (6)$$

Figure 18: Styrke-beregninger

14 Appendix F

14.1 Tekstprompt til OpenAI

"You are an image analysis system trained to make visual estimations from pictures, even when uncertain."

"Always provide your best guess based only on the image. Do NOT say you can't tell."

Return exactly:

1. Object_type (one word)
2. Object_height (in meters) – number only, no units
3. Estimated grip force (N) to lift the object with rubber fingertips and 2.5 cm² contact area – number only, no units
4. Is this object electronic waste? Answer with one word only: 'yes' or 'no'.

Make a guess even if you're not sure. Never reply with an explanation or apology.

15 Appendix G

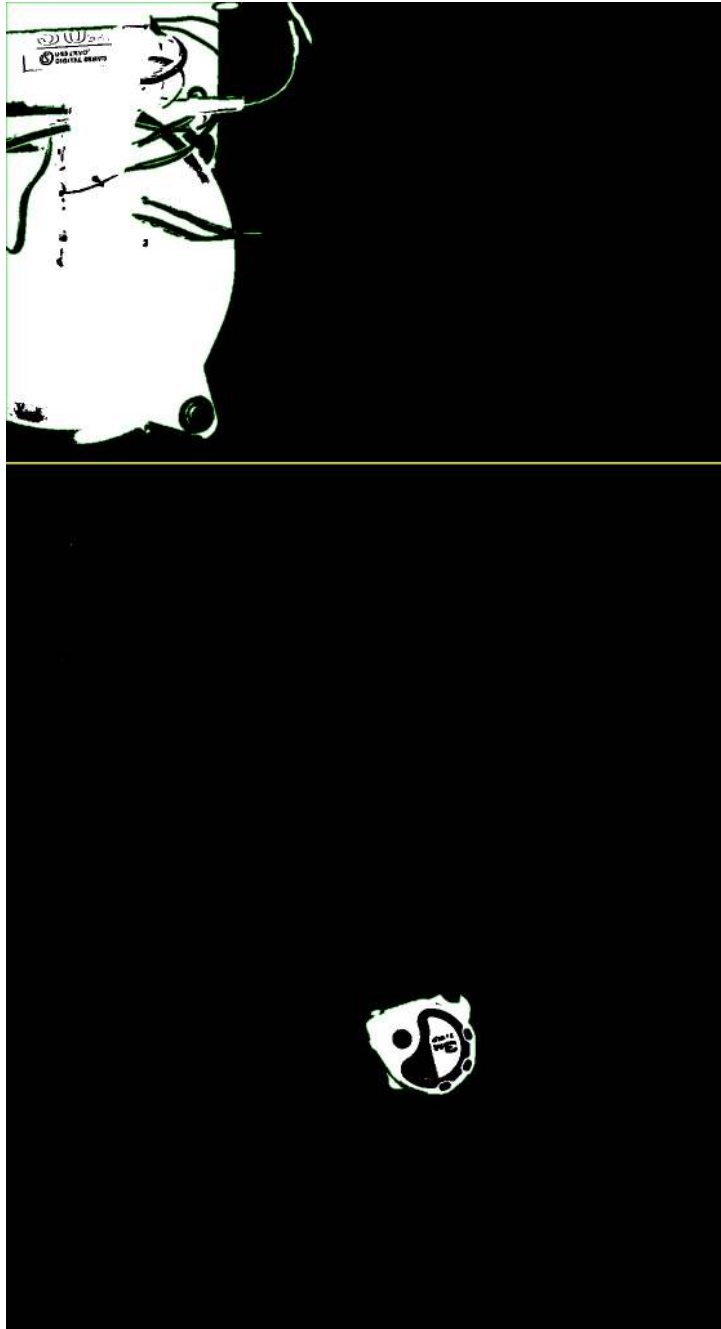


Figure 19: Konturbillede af arbejdsområde og objekt

16 Appendix H

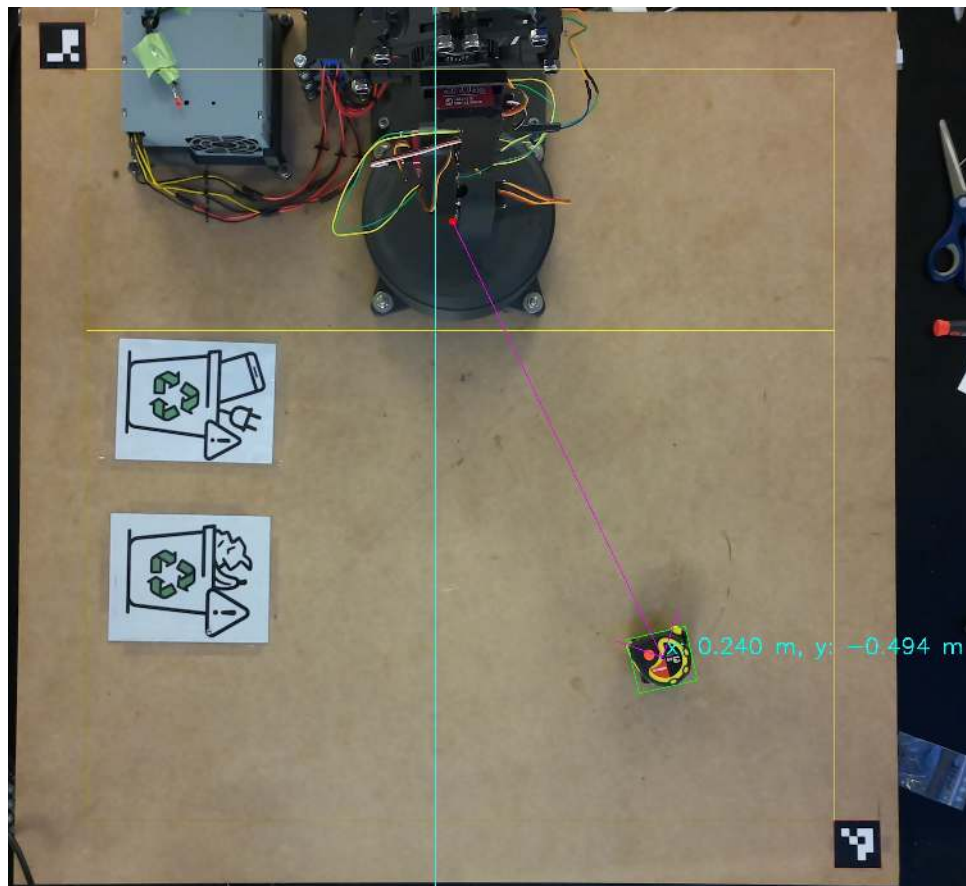


Figure 20: Annoteret billede af arbejdsområde med markerede objekter, afstandsvektor og referencepunkter.